

ARCHITECTURE BY CONSTRUCTION

Verified Blueprints for the Post-Promise Era of TypeScript

CONTENTS

I	Orientation	1
1	How to use this manual	5
1.1	The Active Reading Protocol	5
1.2	Typographic Conventions	5
1.3	The Goal: Architecture by Construction	5
2	Doctrine: The Ten Laws of Effect Architecture	7
2.1	Law 1: The Edge Rule	7
2.2	Law 2: The Composition Root	7
2.3	Law 3: The Empty Signature	8
2.4	Law 4: The Boundary Guard	8
2.5	Law 5: The Scope Guarantee	8
2.6	Law 6: Structured Concurrency	8
2.7	Law 7: The Error Algebra	8
2.8	Law 8: Service Purity	9
2.9	Law 9: The Interop Guard	9
2.10	Law 10: Implementation Opacity	9
3	The Promise Trap (and why Effect exists)	11
3.1	Purpose	11
3.2	Invariants	11
3.3	New concepts	12
3.4	The build	12
3.5	Walkthrough	12
3.5.1	The Lifecycle Split	12
3.5.2	The Eager Promise	12
3.5.3	The Lazy Effect	13
3.6	Checkpoint	13
3.7	Exercises (verifiable)	13
3.8	The Rosetta Stone	14
3.9	Footguns	14

4	The Effect Contract in 30 Minutes	15
4.1	Purpose	15
4.2	Invariants	15
4.3	New concepts	15
4.4	Walkthrough	15
4.4.1	The Three Channels: A, E, R	15
4.4.2	The "Invisibility" of Requirements	16
4.4.3	The Static Guarantee	16
4.5	The Algebraic Contract	16
4.6	Checkpoint	17
4.7	Exercises (verifiable)	17
4.8	Footguns	17
5	Reading Types as Obligations	19
5.1	Purpose	19
5.2	Invariants	19
5.3	New concepts	19
5.4	Walkthrough	19
5.4.1	Error Unification	19
5.4.2	Total Functions vs Partial Functions	20
5.4.3	The Power of Never	20
5.5	Checkpoint	20
5.6	Exercises (verifiable)	20
5.7	Footguns	20
II	The Core Calculus	23
6	Constructors: Bringing the World into Effect	27
6.1	Purpose	27
6.2	The three channels and your first act of discipline	27
6.3	Walkthrough	27
6.3.1	Pure Values: Effect.succeed	27
6.3.2	Synchronous Side-Effects: Effect.sync and Effect.try	27
6.3.3	Asynchronous Bridges: Effect.promise and Effect.tryPromise	28
6.3.4	The "Laziness of try"	28
6.4	"Pure" values: succeed and fail	28
6.5	Synchronous side effects: sync and try	28
6.6	Asynchronous side effects: promise vs tryPromise	29
6.7	Laziness and re-execution: suspend	29
6.8	Exercises	29
7	Sequencing and Control Flow	31
7.1	Purpose	31
7.2	Walkthrough	31
7.2.1	The Generator Pattern (Effect.gen)	31
7.2.2	When to use .pipe()	32

7.2.3	Concurrency: Effect.all	32
7.3	Exercises	32
8	Errors You Mean vs Defects You Don't	33
8.1	Purpose	33
8.2	Invariants	33
8.3	New concepts	33
8.4	Walkthrough	33
8.4.1	The Algebra of Failure	33
8.4.2	Precise Recovery: catchTag	34
8.4.3	Understanding Cause	34
8.5	Checkpoint	35
8.6	Exercises	35
8.7	Footguns	35
III	Requirements & Architecture	37
9	Services: Interfaces that Don't Leak	41
9.1	Purpose	41
9.2	Invariants	41
9.3	New concepts	41
9.4	Walkthrough	41
9.4.1	Service Identity: The Tag	41
9.4.2	Pulling from the Environment	42
9.4.3	Why Tag is Superior	42
9.5	Checkpoint	42
9.6	Exercises	42
9.7	Footguns	43
10	Layers: Constructors for Services	45
10.1	Purpose	45
10.2	Invariants	45
10.3	New concepts	45
10.4	Walkthrough	45
10.4.1	Dependency Graph Topology	45
10.4.2	Layer Lifecycle	46
10.4.3	Sharing and Memoization	46
10.5	Checkpoint	46
10.6	Exercises	47
10.7	Footguns	47
11	Scope: resources, finalizers, and "cleanup that still runs when everything is on fire"	49
11.1	Purpose	49
11.2	Invariants	49
11.3	New concepts	49
11.4	Walkthrough	49

11.4.1	The LIFO Guarantee	50
11.4.2	The canonical primitive: acquireRelease	50
11.4.3	”Just run it safely”: Effect.scoped	50
11.4.4	Layers with lifetimes: scoped services	50
11.5	The Acquisition Timeline	50
11.6	Checkpoint	51
11.7	Exercises	51
11.8	Footguns	51
12	Runtime and execution boundaries: where the blueprint becomes reality	53
12.1	Purpose	53
12.2	Invariants	53
12.3	New concepts	53
12.4	Walkthrough	53
12.4.1	The Interpreter Analogy	54
12.4.2	Execution Boundaries	54
12.4.3	Reusable Environments: ManagedRuntime	54
12.5	Checkpoint	54
12.6	Exercises	54
12.7	Footguns	55
13	Composition roots: one dependency graph, one provide, one run	57
13.1	Purpose	57
13.2	Invariants	57
13.3	New concepts	57
13.4	Walkthrough	57
13.4.1	Dependency Graph Assembly	57
13.4.2	The Main Entry Point	58
13.5	Checkpoint	58
13.6	Exercises	59
13.7	Footguns	59
14	Configuration as a First-Class Program	61
14.1	Purpose	61
14.2	Invariants	61
14.3	New concepts	61
14.4	Walkthrough	61
14.4.1	Nested Configuration	62
14.4.2	Loading the Config	62
14.4.3	Standardizing on ConfigProviders	62
14.5	Checkpoint	62
14.6	Exercises	63
14.7	Footguns	63
15	Schema-First Boundaries	65
15.1	Purpose	65
15.2	Invariants	65

15.3	New concepts	65
15.4	Walkthrough	65
15.4.1	The Checked Zone	65
15.4.2	Defining your domain	66
15.4.3	Beyond Basic Types: Branding	66
15.4.4	The Transformation Pipeline	66
15.4.5	Schema-First Error Handling	67
15.5	Checkpoint	67
15.6	Exercises	67
15.7	Footguns	67
16	One End-to-End Slice	69
16.1	Purpose	69
16.2	Invariants	69
16.3	New concepts	69
16.4	Walkthrough	69
16.4.1	Building a Vertical Slice	69
16.5	Checkpoint	70
16.6	Exercises	71
16.7	Footguns	71
17	The "Rules of the Monorepo"	73
17.1	Purpose	73
17.2	Invariants	73
17.3	New concepts	73
17.4	Walkthrough	73
17.4.1	Dependency Tier Topology	74
17.4.2	The non-negotiables	74
17.4.3	The strict hierarchy of packages	74
17.4.4	Export Rules: What to show	74
17.4.5	Review Checklist for PRs	74
17.5	Checkpoint	75
17.6	Exercises	75
17.7	Footguns	75
IV	Concurrency & Resilience	77
18	Fibers: structured concurrency, cancellation, and not lying about parallelism	81
18.1	The Throughput Engine	81
18.2	Purpose	81
18.3	Invariants	82
18.4	New concepts	82
18.5	Walkthrough	82
18.5.1	The Chain of Command	82
18.5.2	Automatic Interruption	83
18.5.3	When to use forkDaemon	83

18.5.4 Manual Control: Fiber.interrupt	83
18.6 The Runtime Loop	83
18.7 Checkpoint	84
18.8 Exercises	84
18.9 Footguns	84
19 Resilience: retries, timeouts, and schedules as policy	85
19.1 Purpose	85
19.2 Invariants	85
19.3 New concepts	85
19.4 Walkthrough	85
19.4.1 Policy Algebra: Composable Resilience	85
19.4.2 Timeout as an Operational State	86
19.5 Checkpoint	86
19.6 Exercises	87
19.7 Footguns	87
20 Advanced Concurrency Patterns	89
20.1 Purpose	89
20.2 Concurrency Archetypes	89
21 Coordination primitives: building deterministic concurrency graphs	91
21.1 Purpose	91
21.2 Invariants	91
21.3 New concepts	91
21.4 Walkthrough	91
21.4.1 Backpressure as a Safety Valve	92
21.4.2 When to use PubSub	92
21.4.3 Semaphores: Protecting Fragile Resources	92
21.5 Checkpoint	92
21.6 Exercises	93
21.7 Footguns	93
22 Streams: "Effect, but many values"	95
22.1 Purpose	95
22.2 Invariants	95
22.3 New concepts	95
22.4 Walkthrough	95
22.4.1 The Pull Model: A Stream Vacuum	95
22.4.2 The canonical pipeline	96
22.4.3 Memory-Efficient I/O	96
22.4.4 Resource Safety	96
22.5 Checkpoint	96
22.6 Exercises	97
22.7 Footguns	97
23 State + Backpressure primitives	99

23.1	Purpose	99
23.2	Invariants	99
23.3	New concepts	99
23.4	Walkthrough	99
23.4.1	Ref: Controlled Shared State	99
23.4.2	The Race Condition Hazard	100
23.4.3	SynchronizedRef: Atomic Effectful Updates	100
23.4.4	State Lifecycle	100
23.5	Checkpoint	100
23.6	Exercises	100
23.7	Footguns	101
V	Observability & Verification	103
24	Observability: logging and tracing as part of the program's semantics	107
24.1	Purpose	107
24.2	Invariants	107
24.3	New concepts	107
24.4	Walkthrough	107
24.4.1	Contextual Inheritance	108
24.4.2	Spans: The "Navigation Map"	108
24.5	Checkpoint	108
24.6	Exercises	108
24.7	Footguns	109
25	Observability as semantics	111
25.1	Purpose	111
25.2	Invariants	111
25.3	New concepts	111
25.4	Walkthrough	111
25.4.1	Metrics: answering "How Much"	112
25.4.2	Latency Histograms	112
25.4.3	Zero-Overhead Semantic Observability	112
25.5	Checkpoint	112
25.6	Exercises	112
25.7	Footguns	113
26	Effect Platform: the "edge algebra"	115
26.1	Purpose	115
26.2	Invariants	115
26.3	New concepts	115
26.4	Walkthrough	115
26.4.1	Subprocess Pipelines	115
26.4.2	Controlled Execution	116
26.4.3	Interruption Propagation	116
26.5	Checkpoint	116

26.6 Exercises	116
26.7 Footguns	116
27 Platform Runtime	119
27.1 Purpose	119
27.2 Invariants	119
27.3 New concepts	119
27.4 Walkthrough	119
27.4.1 Standardizing on abstractions	119
27.4.2 Platform-specific wiring	120
27.5 Checkpoint	120
27.6 Exercises	120
27.7 Footguns	120
28 Testing: make time, resources, and failures deterministic	123
28.1 Purpose	123
28.2 Invariants	123
28.3 New concepts	123
28.4 Walkthrough	123
28.4.1 Standardizing on @effect/vitest	123
28.4.2 Deep Mocking: Swapping layers	124
28.4.3 Controlling Time with TestClock	124
28.5 Checkpoint	124
28.6 Exercises	125
28.7 Footguns	125
VI The Horizon	127
29 The Way of the Architect: Toward a Post-Promise Era	131
29.1 Purpose	131
29.2 Invariants	131
29.3 The Horizon Concepts	131
29.4 Walkthrough: The Architecture of the Next Decade	132
29.4.1 The AI-Human Hybrid Lifecycle	132
29.4.2 Post-Framework Composition	132
29.5 The Architect's Manifesto	133
30 Glossary of Intent	135
Addendum: Essential Resources	137
Index	143

Tier I

ORIENTATION



Blueprint Overview

CONTENTS

1	How to use this manual	5
1.1	The Active Reading Protocol	5
1.2	Typographic Conventions	5
1.3	The Goal: Architecture by Construction	5
2	Doctrine: The Ten Laws of Effect Architecture	7
2.1	Law 1: The Edge Rule	7
2.2	Law 2: The Composition Root	7
2.3	Law 3: The Empty Signature	8
2.4	Law 4: The Boundary Guard	8
2.5	Law 5: The Scope Guarantee	8
2.6	Law 6: Structured Concurrency	8
2.7	Law 7: The Error Algebra	8
2.8	Law 8: Service Purity	9
2.9	Law 9: The Interop Guard	9
2.10	Law 10: Implementation Opacity	9
3	The Promise Trap (and why Effect exists)	11
3.1	Purpose	11
3.2	Invariants	11
3.3	New concepts	12
3.4	The build	12
3.5	Walkthrough	12
3.5.1	The Lifecycle Split	12
3.5.2	The Eager Promise	12
3.5.3	The Lazy Effect	13
3.6	Checkpoint	13
3.7	Exercises (verifiable)	13
3.8	The Rosetta Stone	14
3.9	Footguns	14
4	The Effect Contract in 30 Minutes	15
4.1	Purpose	15
4.2	Invariants	15
4.3	New concepts	15
4.4	Walkthrough	15
4.4.1	The Three Channels: A, E, R	15
4.4.2	The "Invisibility" of Requirements	16
4.4.3	The Static Guarantee	16

Chapter 1

HOW TO USE THIS MANUAL

This is not a traditional programming book. It is a **Living Manual**. Every line of code you see in these pages exists as a compiled, type-checked, and verified unit in the companion repository.

1.1 THE ACTIVE READING PROTOCOL

To get the most out of this manual, you should not read it in isolation. You should have the repository open in your IDE.

1. **Locate the Source:** Most code blocks are pulled directly from `packages/bible-examples/src`. When you see a snippet, find the file.
2. **Run the Capture:** Use `pnpm run:capture` to see the terminal output generated by the examples.
3. **Break the Invariants:** Deliberately violate one of the "Ten Laws" in the code and observe the TypeScript compiler—and the Effect runtime—respond.

1.2 TYPOGRAPHIC CONVENTIONS

- `Effect<A, E, R>`: Technical types and signatures.
- **Fiber**: Fundamental concepts defined in the glossary.
- `Effect.runPromise`: Specific API calls.

1.3 THE GOAL: ARCHITECTURE BY CONSTRUCTION

By the end of this manual, you will not just "know" Effect. You will have built a mental model that treats software as a series of nested descriptions (blueprints) that are only executed when they are safe, provided, and supervised.

Chapter 2

DOCTRINE: THE TEN LAWS OF EFFECT ARCHITECTURE

In the Effect ecosystem, we do not follow “best practices” because they are popular; we follow them because they provide mathematical guarantees about our system’s behavior. Violation of these laws results in “State Rot”—a condition where the types say one thing, but the runtime does another.

2.1 LAW 1: THE EDGE RULE

LAW 1

A library module or business domain file must never call `Effect.run*`.

THE SPIRIT of this law is Preservation of Control. When you keep a computation as an *Effect*, you preserve its metadata. The moment you run it, you “flatten” that metadata into a primitive *Promise* or a value. Be humble: export the description, let the consumer choose the execution.

2.2 LAW 2: THE COMPOSITION ROOT

LAW 2

A program should have exactly one entry point that provides the environment and executes.

This is about Context Clarity. Law 2 demands that the “Shape of the World” is defined in exactly one file (*main.ts*). This makes your system “Environment Transparent”—you can look at one file and see every database, logger, and API client being used.

2.3 LAW 3: THE EMPTY SIGNATURE

LAW 3

Functions returning `void`, `Promise<void>`, or `boolean` are usually a design error.

We seek Total Information. A `void` return is a "blind spot"—it tells you the function is done, but nothing about the result. A `boolean` is a "Binary Lie"—it forces branches without explaining why. Success should return data, and failure should return an Error Algebra.

2.4 LAW 4: THE BOUNDARY GUARD

LAW 4

Parse at the edge; use typed domain models internally.

*This is Boundary Integrity. Law 4 enforces a "Checked Zone." Once data passes through a **Schema** at the API or Database edge, it is no longer just "data"—it is a Domain Object guaranteed to be valid. Your logic can then focus on business value rather than defensive null-checking.*

2.5 LAW 5: THE SCOPE GUARANTEE

LAW 5

Resource lifetime must be managed by the Environment, not by manual cleanup.

*The Spirit is Guaranteed Finalization. Human developers are bad at remembering to call `.close()` or `.disconnect()`. By moving resource management into **Scope**, we make cleanup "invisible and inevitable."*

2.6 LAW 6: STRUCTURED CONCURRENCY

LAW 6

Fibers must be born and die within the scope of their parent.

This is about Logical Supervision. Every concurrent task should have a parent that knows how to wait for it or kill it. Law 6 prevents "Orphan Fibers" that consume memory invisibly.

2.7 LAW 7: THE ERROR ALGEBRA

LAW 7

Never catch `All` unless you are at the Composition Root.

This is Honesty of Failure. Law 7 demands that errors remain "visible values" until they reach the edge of the system. Swallowing errors is a "sin of silence" that makes debugging production systems nearly impossible.

2.8 LAW 8: SERVICE PURITY

LAW 8

The Environment (R) is for static services; the Success channel (A) is for dynamic data.

The Spirit is Environmental Statelessness. The Context should be a "Toolbox," not a "Bag of State." If you put the "Current User ID" in the context, you cannot easily process two users at once.

2.9 LAW 9: THE INTEROP GUARD

LAW 9

Never wrap an Effect in an `async` function unless crossing a framework boundary.

This is Laziness Protection. When you wrap an Effect in `async`, you "jail" the Effect inside a running Promise, losing the ability to retry, interrupt, or provide context. Keep your code "Effect-Native" as deep as possible.

2.10 LAW 10: IMPLEMENTATION OPACITY

LAW 10

Services must be defined by their interface, never by their implementation classes.

The Spirit is Abstract Dependency. If your business logic imports `PostgresUserRepository`, you are coupled to a database driver. Law 10 ensures your core logic remains "Implementation Blind."

CANONICAL REFERENCE

- Effect Code Style Guidelines
- Service Pattern

Chapter 3

THE PROMISE TRAP (AND WHY EFFECT EXISTS)

3.1 PURPOSE

The transition from standard TypeScript async programming to Effect is often motivated by a growing sense of unease. In a “normal” production codebase, we frequently accept certain failure modes as inevitable. We accept that tests must mock imports to isolate logic. We accept that a function signature like `(id: string) => Promise<User>` tells us almost nothing about what the function actually *does*—does it read from a database? Does it log to the console? Does it modify global state?

The purpose of this chapter is to establish these concrete failure modes not as “how things are,” but as the “Promise Trap”: a set of architectural compromises forced upon us by the Promise abstraction itself. Specifically, we will examine:

- **Implicit Dependencies:** How functions hide their requirements (network, time, config) inside their implementation, making them hard to test and reuse.
- **Invisible Effects:** How side effects happen “automatically” upon function invocation, stripping us of control over *when* and *how* they run.
- **The Mocking Spiral:** How the inability to pass dependencies explicitly leads to fragile, module-mutating test harnesses.

Our goal is not to moralize about functional purity. It is to make you feel the operational cost of these compromises precisely. Then, we will introduce the primary antidote Effect offers immediately: the concept that **effects are values**. By turning interactions with the world into data structures, we regain control over execution, making it explicit, lazy, and composable.

3.2 INVARIANTS

To escape the Promise Trap, we must adopt a new set of mental invariants. These are the hard rules that distinguish idiomatic Effect code from “Promise code wrapped in Effect.”

1. **No Implicit Inputs:** If code performs I/O, reads the current time, generates randomness, or accesses process state, that is a dependency. It must be declared. In standard TypeScript, these are often invisible; in Effect, they become part of the type signature.
2. **No Implicit Execution:** Merely calling a function should not trigger a side effect. Execution must be deferred until the "end of the world" (the application boundary). If execution begins "somewhere in the middle"—on import, inside a constructor, or implicitly when a promise is created—you have lost control of the runtime characteristics of your program.
3. **Testing Real Code:** If your tests rely on swizzling imports (e.g., `jest.mock` or `vi.mock`) to simulate different environments, you are essentially testing the test harness, not your program. A robust architecture allows you to swap implementations (dependencies) without mutating the JavaScript module system.

3.3 NEW CONCEPTS

- "Implicit dependency" as a first-class defect category (pre-Effect).
- "Make execution explicit" as the primary win (the first Effect superpower).

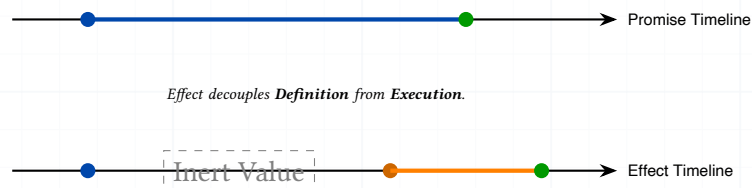
3.4 THE BUILD

- `src/ch01/before.ts`
- `src/ch01/after.ts` (the same logic, but made explicitly executable)

3.5 WALKTHROUGH

3.5.1 The Lifecycle Split

The fundamental difference between a Promise and an Effect is *when* the work happens.



3.5.2 The Eager Promise

Consider this standard TypeScript code for calculating a price. The moment you call `getPrice`, the side-effect (a database call or network request) has already started.

```
// A standard Promise-based function
const getPrice = (id: string): Promise<number> => {
  console.log(`Fetching price for ${id}...`);
  return fetch(`/api/prices/${id}`).then(r => r.json());
}

// THE TRAP: This console.log happens NOW.
```

```
const pricePromise = getPrice("apple");
```

If you want to retry this operation because the network is flaky, you cannot use the `pricePromise` variable. It's already running (or failed). You must wrap the *call itself* in a new function, creating a "callback hell" of retry logic.

3.5.3 The Lazy Effect

Now, look at the Effect equivalent. `getPrice` returns a value that *describes* the action, but performs no work.

```
import { Effect } from "effect";

const getPrice = (id: string) =>
  Effect.log(`Fetching price for ${id}...`).pipe(
    Effect.flatMap(() => Effect.tryPromise(() => fetch(`/api/prices/${id}`))),
    Effect.flatMap(res => Effect.tryPromise(() => res.json()))
  );

// NOTHING happens here. No logs, no fetch.
const priceBlueprint = getPrice("apple");
```

Because `priceBlueprint` is just a value, we can transform it before it ever runs. We can add a timeout, a retry policy, and logging without changing a single line of the original function.

```
const resilientPrice = priceBlueprint.pipe(
  Effect.timeout("2 seconds"),
  Effect.retry({ times: 3 })
);
```

This is the power of the "Blueprint Model": you separate the **Logic** (how to calculate a price) from the **Strategy** (how many times to retry if it fails).

3.6 CHECKPOINT

Add a tiny runner:

```
// apps/book-app/src/ch01/run.ts
import { Effect } from "effect"
import { getPrice } from "../after"

Effect.runPromise(getPrice(100)).then(console.log)
```

Expected behavior: this file is the first place execution necessarily occurs (not at import time). That separation is a best practice: "run effects at the outermost parts of your application."

3.7 EXERCISES (VERIFIABLE)

1. Convert one additional implicit dependency into an explicit Effect:

- `reading process.env`
- `time (Date.now())`
- `randomness (Math.random())`

Write a test that proves "nothing runs until `runPromise` is called."

2. Failure discipline: Make `isEnabled` fail with a typed, meaningful error (don't throw raw).

3.8 THE ROSETTA STONE

A direct comparison between the "Imperative Hope" of Legacy TypeScript and the "Verified Intent" of Effect.

Feature	Legacy (Promise)	Effect
Execution	Eager: Starts immediately upon creation, leading to race conditions.	Lazy: Just a blueprint of intent. Nothing happens until explicitly run.
Errors	Hidden: Can throw anywhere. Requires manual try/catch blocks.	Explicit: Typed in the E channel. Total information about failure.
Context	Managed through global state or fragile prop-drilling.	Context-bound: Provided via the R channel with type-safety.
Cleanup	Manual <code>finally</code> blocks. Prone to resource leaks.	Automatic: Managed by Scope with deterministic LIFO order.
Concurrency	Unbounded: Hard to limit or monitor effectively.	Structured: First-class support for limits, timeouts, and races.

3.9 FOOTGUNS

...

CANONICAL REFERENCE

- Introduction to Effect
- Running Effects

Chapter 4

THE EFFECT CONTRACT IN 30 MINUTES

4.1 PURPOSE

Make the reader operationally fluent with the core contract: an Effect is a description of computation with explicit success, explicit expected failure, and explicit requirements. Then teach the rule: "execution is explicit and usually happens once at the boundary."

4.2 INVARIANTS

1. Effects are values: building them is not running them.
2. Expected errors live in the Error channel, not in "maybe throws."
3. Requirements are declared at the type level (and satisfied by providing services/layers later).

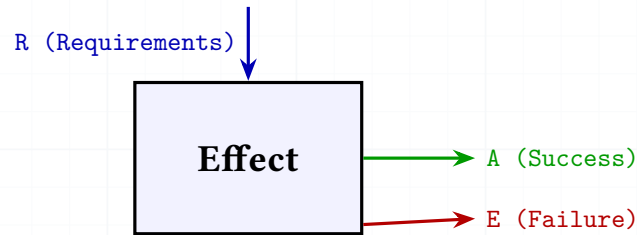
4.3 NEW CONCEPTS

- `Effect<Success, Error, Requirements>` meaning.
- Constructors: `Effect.sync` and `Effect.try` for sync side effects.
- Runners: `runPromise` as the default boundary runner.

4.4 WALKTHROUGH

4.4.1 The Three Channels: A, E, R

Every Effect signature tells a complete story about its behavior.

*The Blueprint Contract*

Every Effect signature tells a complete story about its behavior.

- **A (Success)**: What the program produces if everything goes right.
- **E (Error)**: What the program produces if it fails in an expected way.
- **R (Requirements)**: What the program needs from the environment to run.

```
// Signature: Effect<User, DbError, Database>
const loadUser = (id: string) => ...
```

This signature is an *Honest Contract*. It says: "If you give me a Database, I will eventually give you a User, or I might fail with a DbError."

4.4.2 The "Invisibility" of Requirements

In traditional dependency injection, you pass services through constructors or arguments. In Effect, requirements are "bubbled up" automatically by the type system.

```
const program = Effect.gen(function* () {
  const user = yield* loadUser("123") // Needs Database
  const config = yield* loadConfig() // Needs Config
  return { user, config }
})

// Resulting type: Effect<{user, config}, DbError | ConfigError,
// Database | Config>
```

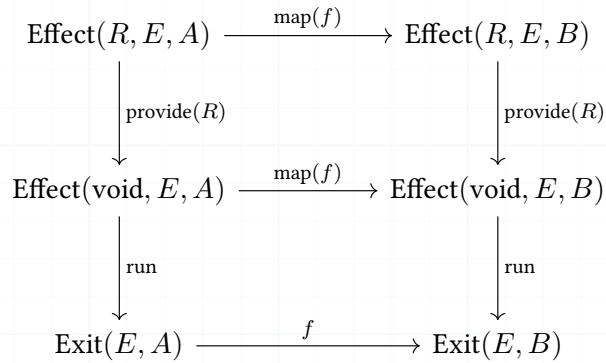
Notice how the R channel automatically combined Database and Config. This is the "Unify" principle: Effect handles the complexity of merging dependencies so you don't have to.

4.4.3 The Static Guarantee

Because requirements are tracked at the type level, it is impossible to run a program that is missing a dependency.

4.5 THE ALGEBRAIC CONTRACT

The Effect contract is mathematically verifiable. We can visualize this as a commutative diagram where every path to execution is guarded by the same invariants.



This diagram represents the **Preservation of Intent**. Whether you transform the value before or after providing requirements, the resulting *Exit* state remains consistent. In the Post-Promise era, we don't just "hope" the types align; we prove they do.

4.6 CHECKPOINT

Run `src/ch02/running.ts`. Expect:

- "before" prints 0
- "after" prints 1
- execution occurs only after `runPromise`.

4.7 EXERCISES (VERIFIABLE)

1. Replace a thrown exception with `Effect.try` so the error is captured in the Error channel.
2. Write a test that fails if `counter` changes before the runner executes the effect.

4.8 FOOTGUNS

- Calling `Effect.runPromise` inside constructors (destroys composability).
- Confusing the generator authoring style with runtime semantics.

CANONICAL REFERENCE

- The Effect Type
- Creating Effects
- Using Generators

Chapter 5

READING TYPES AS OBLIGATIONS

5.1 PURPOSE

Make the reader fluent in "type-driven obligation accounting." You should be able to look at an Effect type and answer three questions immediately:

1. What does it produce on success?
2. What failures are expected (and therefore handled)?
3. What does it require from its environment?

5.2 INVARIANTS

1. If a program "needs something," it should appear in Requirements.
2. If `Error = never`, you are asserting it cannot fail with expected errors (strong claim).
3. If you can't describe the requirements in one sentence, your effect is doing too much.

5.3 NEW CONCEPTS

- Requirements parameter as declared dependency set.
- Context/service mental model (preview; full details later).

5.4 WALKTHROUGH

5.4.1 Error Unification

In many systems, you lose type information about errors as they bubble up (e.g., `try/catch` blocks catch any). In Effect, errors are unified into a discriminated union.

```
const validate = (data: unknown) => ... // Effect<void, ValidationError>  
const save = (data: Data) => ... // Effect<void, DbError>
```

```
const flow = (input: unknown) => Effect.gen(function* () {
  const data = yield* validate(input)
  yield* save(data)
})

// Type: Effect<void, ValidationError | DbError>
```

The pipe symbol (`|`) in the error channel is an **Obligation List**. The type system is telling you: "If you want to run this, you must be prepared to handle both a `ValidationError` and a `DbError`."

5.4.2 Total Functions vs Partial Functions

A standard JS function is "partial"—it might throw an error that isn't in its signature. An Effect is "total"—it explicitly models its failure surface.

```
// PARTIAL: Might throw, might return string
function get(id: string): string { ... }

// TOTAL: Explicitly models success, failure, and needs
function get(id: string): Effect<string, NotFound, DbClient> { ... }
```

5.4.3 The Power of Never

- `Effect<A, never, R>`: A program that *cannot* fail with an expected error.
- `Effect<A, E, never>`: A program that is *self-contained* and needs no dependencies.

By using `never` correctly, you provide valuable hints to the compiler and your teammates about the safety and complexity of your code.

5.5 CHECKPOINT

For this chapter, the checkpoint is compile-time:

- You should be able to label Success/Error/Requirements in the drills without running anything.

5.6 EXERCISES (VERIFIABLE)

1. Write 5 effects of increasing complexity and annotate their obligations (S/E/R).
2. Write one effect where `Error = never` is incorrect, then fix it so the error is explicit.

5.7 FOOTGUNS

- Using `any` to "make the type go away." You've destroyed the obligation accounting.
- Treating `Error = unknown` as a default: you've opted out of meaningful failure types.

CANONICAL REFERENCE

- The Effect Type
- Managing Services

Tier II

THE CORE CALCULUS



Blueprint Overview

CONTENTS

6	Constructors: Bringing the World into Effect	27
6.1	Purpose	27
6.2	The three channels and your first act of discipline	27
6.3	Walkthrough	27
6.3.1	Pure Values: Effect.succeed	27
6.3.2	Synchronous Side-Effects: Effect.sync and Effect.try	27
6.3.3	Asynchronous Bridges: Effect.promise and Effect.tryPromise	28
6.3.4	The "Laziness of try"	28
6.4	"Pure" values: succeed and fail	28
6.5	Synchronous side effects: sync and try	28
6.6	Asynchronous side effects: promise vs tryPromise	29
6.7	Laziness and re-execution: suspend	29
6.8	Exercises	29
7	Sequencing and Control Flow	31
7.1	Purpose	31
7.2	Walkthrough	31
7.2.1	The Generator Pattern (Effect.gen)	31
7.2.2	When to use .pipe()	32
7.2.3	Concurrency: Effect.all	32
7.3	Exercises	32
8	Errors You Mean vs Defects You Don't	33
8.1	Purpose	33
8.2	Invariants	33
8.3	New concepts	33
8.4	Walkthrough	33
8.4.1	The Algebra of Failure	33
8.4.2	Precise Recovery: catchTag	34
8.4.3	Understanding Cause	34
8.5	Checkpoint	35
8.6	Exercises	35
8.7	Footguns	35

Chapter 6

CONSTRUCTORS: BRINGING THE WORLD INTO EFFECT

6.1 PURPOSE

You must choose (deliberately) what enters the success channel, what enters the error channel, and what becomes an "unexpected defect." This chapter is a guided tour of the core constructors.

6.2 THE THREE CHANNELS AND YOUR FIRST ACT OF DISCIPLINE

An `Effect<A, E, R>` is a computation that, when run, may:

- succeed with `A`
- fail with `E`
- require `R`

But there is a fourth thing it may do: die (defect). Unexpected failures are bugs, invariants broken, integration lies, and cosmic rays.

6.3 WALKTHROUGH

6.3.1 Pure Values: `Effect.succeed`

Use `Effect.succeed` to wrap a value that is already calculated and has no side-effects.

```
| const one = Effect.succeed(1) // Effect<number, never, never>
```

6.3.2 Synchronous Side-Effects: `Effect.sync` and `Effect.try`

If you have a function that performs a side-effect (like reading from `localStorage` or generating a random number), wrap it in `Effect.sync`.

```
const random = Effect.sync(() => Math.random())
```

If the synchronous function might throw an exception, use `Effect.try`. This moves the error into the `E` channel.

```
const parse = (json: string) =>
  Effect.try({
    try: () => JSON.parse(json),
    catch: (e) => new JsonParseError({ message: String(e) })
  })
```

6.3.3 Asynchronous Bridges: `Effect.promise` and `Effect.tryPromise`

To bring existing Promise-based code into Effect, use the promise constructors.

```
// Use tryPromise when the fetch might fail (e.g., network error)
const fetchData = (url: string) =>
  Effect.tryPromise({
    try: () => fetch(url).then(res => res.json()),
    catch: (e) => new NetworkError({ message: String(e) })
  })
```

6.3.4 The "Laziness of try"

A critical distinction: `Effect.tryPromise` does **not** call the promise immediately. It stores the *function* that produces the promise. This preserves the laziness contract of the entire system.

6.4 "PURE" VALUES: SUCCEED AND FAIL

```
import { Effect } from "effect"

const parsePort = (s: string) =>
  Number.isInteger(Number(s))
    ? Effect.succeed(Number(s))
    : Effect.fail(new Error(`Invalid port: ${s}`))
```

6.5 SYNCHRONOUS SIDE EFFECTS: SYNC AND TRY

- `Effect.sync(thunk)` for synchronous work that is not expected to throw.
- `Effect.try(...)` for synchronous work that might throw.

```
import { Effect } from "effect"

const now = Effect.sync(() => Date.now())

const parseJson = (s: string) =>
  Effect.try({
    try: () => JSON.parse(s) as unknown,
    catch: (e) => new Error(`Bad JSON: ${String(e)}`)
  })
```

6.6 ASYNCHRONOUS SIDE EFFECTS: PROMISE VS TRYPROMISE

- Use `Effect.promise` only when you are confident the promise cannot reject (or you truly want rejection to be a defect).
- Use `Effect.tryPromise` when rejection is part of the expected operational surface.

```
import { Effect } from "effect"

const fetchText = (url: string) =>
  Effect.tryPromise({
    try: () => fetch(url).then((r) => r.text()),
    catch: (e) => new Error(`fetch failed: ${String(e)}`)
  })
```

6.7 LAZINESS AND RE-EXECUTION: SUSPEND

If you write `Effect.succeed(i++)`, the increment happens immediately. Use `Effect.suspend` to defer evaluation.

```
import { Effect } from "effect"

let i = 0
const bad = Effect.succeed(i++) // evaluated now
const good = Effect.suspend(() => Effect.succeed(i++)) // evaluated on
  each run
```

6.8 EXERCISES

1. Wrap `fs.readFile` in an effect that models failures in the error channel.
2. Write a function `env(name: string)` that reads `process.env[name]` and fails if missing.

CANONICAL REFERENCE

- Creating Effects

Chapter 7

SEQUENCING AND CONTROL FLOW

7.1 PURPOSE

Once you have mastered the art of constructing individual effects—bottling lightning, as it were—the next challenge is arranging them into a coherent sequence. In the world of standard Promises, sequencing is implicit: you call a function, it runs, and you `await` the result. This model is convenient but dangerous, as it conflates the *definition* of a workflow with its *execution*.

Effect restores the separation between description and execution. Sequencing in Effect is not just about “doing one thing after another”; it is about composing descriptions of dependencies. We have two primary tools for this:

1. **Generators** (`Effect.gen`): The idiomatic, imperative style that resembles `async/await` but remains purely declarative.
2. **Pipelines** (`pipe`): The functional, data-flow style that emphasizes transformation and composition.

This chapter explores both styles, explaining when to use which, and proving that under the hood, they are the same algebra.

7.2 WALKTHROUGH

7.2.1 The Generator Pattern (`Effect.gen`)

The modern, idiomatic way to sequence Effects is using TypeScript Generators. This allows you to write asynchronous logic that looks like synchronous code, while maintaining full type safety and cancellation support.

```
import { Effect } from "effect"

const program = Effect.gen(function* () {
  // 1. Yield* pulls the A value from the Effect
  const user = yield* loadUser("123")
```

```
// 2. The next line only runs if user succeeds
const config = yield* loadConfig(user.role)

// 3. Normal JS logic works alongside yield*
if (config.debug) {
  yield* Effect.log("Debug mode enabled")
}

return { user, config }
})
```

7.2.2 When to use .pipe()

While `gen` is best for business logic, `.pipe()` is better for “decorating” an existing `Effect` with operators like `retry`, `timeout`, or `provide`.

```
const resilientProgram = program.pipe(
  Effect.retry({ times: 3 }),
  Effect.timeout("10 seconds")
)
```

7.2.3 Concurrency: `Effect.all`

To run multiple `Effects` in parallel, use `Effect.all`. You can control the level of concurrency easily.

```
const downloadAll = Effect.all(urls.map(download), {
  concurrency: 5 // Run 5 at a time
})
```

This is the “Algebra of Sequencing”: you use `gen` for vertical logic (steps) and `all` for horizontal logic (parallelism).

7.3 EXERCISES

1. **Refactor to Generators:** Take a nested `Promise` chain (e.g., fetch user, then fetch posts, then save to DB) and rewrite it using `Effect.gen`. Observe how the nesting disappears.
2. **Pipeline Transformation:** Create an effect that produces a list of numbers. Use `pipe` to filter for even numbers, map them to strings, and join them with commas. Do this without `Effect.gen`.
3. **The Error Short-Circuit:** Write a generator that yields three effects. Make the second one fail. Verify that the third one never runs.

CANONICAL REFERENCE

- Generators
- Pipelines
- Concurrency Options

Chapter 8

ERRORS YOU MEAN VS DEFECTS YOU DON'T

8.1 PURPOSE

Effect makes a hard distinction between "expected errors" (typed, recoverable) and "unexpected errors / defects" (untyped, not part of normal control-flow). This chapter teaches you how to design a failure surface that guides your callers rather than confusing them.

8.2 INVARIANTS

- **E is for Data:** The error channel should contain values, not just exceptions.
- **Requirements are Sticky:** Requirements do not change when recovering from an error unless explicitly provided.
- **Defects are Final:** A defect represents a bug or a fatal system failure; normal business logic should not attempt to "catch" it.

8.3 NEW CONCEPTS

- **Tagged Error:** A class or object with a unique `_tag` property used for discriminated unions.
- **Catching:** Recovering from a specific error value and returning a new Effect.
- **Defect:** An untracked, unrecoverable failure (the Effect "dies").
- **Cause:** The full underlying tree of failure, including defects and interruptions.

8.4 WALKTHROUGH

8.4.1 The Algebra of Failure

Errors in Effect are not just messages; they are data. By using `Data.TaggedError`, you create an "algebra"—a set of values that can be combined and differentiated by the type system.

```
import { Data, Effect } from "effect"

export class UserNotFound extends Data.TaggedError("UserNotFound")<{
  readonly userId: string
}> {}

export class InsufficientFunds extends Data.TaggedError("InsufficientFunds")<{
  readonly required: number
  readonly actual: number
}> {}

type PurchaseError = UserNotFound | InsufficientFunds
```

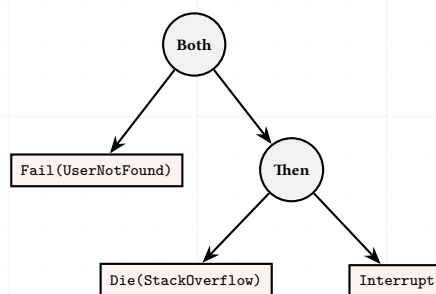
8.4.2 Precise Recovery: catchTag

Because your errors are tagged, you can recover from them with absolute precision. The compiler will ensure you only handle the tags that actually exist in the error channel.

```
const program = purchaseProduct("apple").pipe(
  Effect.catchTag("UserNotFound", (e) =>
    Effect.logWarning(`Prompting login for ${e.userId}`).pipe(
      Effect.flatMap(() => showLoginScreen)
    )
  )
)
```

8.4.3 Understanding Cause

When an Effect fails, it doesn't just produce an error value; it produces a **Cause**. A Cause is a recursive tree structure that tracks the total context of a failure.



A Cause tracks:

- **Failures:** Expected errors in the E channel.
- **Defects:** Unexpected exceptions or crashes.
- **Interruptions:** When a fiber was forced to stop.

Use `Effect.catchAllCause` if you need to log the full breakdown of why a program stopped, including the stack trace of a defect.

8.5 CHECKPOINT

- Expected errors are part of your function signature (E).
- Unexpected errors are defects and are not tracked.
- Use `Data.TaggedError` to build an error algebra.
- Use `catchTag` to recover from specific errors precisely.

8.6 EXERCISES

1. Refactor a Promise API wrapper to return a tagged union error algebra.
2. Replace a `catchAll` that swallows everything with `catchTag` handlers.
3. Use `Effect.catchAllCause` to observe a defect without attempting to recover from it.

8.7 FOOTGUNS

- **Error Swallowing:** Using `catchAll` to return a default value deep in the logic instead of handling specific tags.
- **Untagged Unions:** Returning raw strings or generic `Error` objects makes it impossible for callers to differentiate failures.
- **Sync/Async Mismatch:** Forgetting that `Effect.try` might be needed to capture errors from third-party synchronous code.

CANONICAL REFERENCE

- Expected Errors vs Defects
- Yieldable Errors

Tier III

REQUIREMENTS & ARCHITECTURE



Blueprint Overview

CONTENTS

9	Services: Interfaces that Don't Leak	41
9.1	Purpose	41
9.2	Invariants	41
9.3	New concepts	41
9.4	Walkthrough	41
9.4.1	Service Identity: The Tag	41
9.4.2	Pulling from the Environment	42
9.4.3	Why Tag is Superior	42
9.5	Checkpoint	42
9.6	Exercises	42
9.7	Footguns	43
10	Layers: Constructors for Services	45
10.1	Purpose	45
10.2	Invariants	45
10.3	New concepts	45
10.4	Walkthrough	45
10.4.1	Dependency Graph Topology	45
10.4.2	Layer Lifecycle	46
10.4.3	Sharing and Memoization	46
10.5	Checkpoint	46
10.6	Exercises	47
10.7	Footguns	47
11	Scope: resources, finalizers, and "cleanup that still runs when everything is on fire"	49
11.1	Purpose	49
11.2	Invariants	49
11.3	New concepts	49
11.4	Walkthrough	49
11.4.1	The LIFO Guarantee	50
11.4.2	The canonical primitive: acquireRelease	50
11.4.3	"Just run it safely": Effect.scoped	50
11.4.4	Layers with lifetimes: scoped services	50
11.5	The Acquisition Timeline 40	50
11.6	Checkpoint	51
11.7	Exercises	51
11.8	Footguns	51

Chapter 9

SERVICES: INTERFACES THAT DON'T LEAK

9.1 PURPOSE

This chapter is the architectural hinge: it's where Effect stops being "a better Promise" and becomes "a system for building systems." You will learn how to move dependencies out of function arguments and into the type-safe R channel.

9.2 INVARIANTS

- **Unique Tag IDs:** Every service Tag must have a globally unique string identifier to prevent context collisions.
- **R=never in Methods:** Methods defined within a service interface should typically have R=never. Dependencies of the service are handled by the **Layer** that constructs it.
- **No Manual Constructors:** Services should never be instantiated manually inside business logic; they are provided by the Composition Root.

9.3 NEW CONCEPTS

- **Service:** A set of related functions or data that your program depends on.
- **Context:** An immutable map that holds the current implementations of all required services.
- **Tag:** A unique identifier used to look up a service in the Context.

9.4 WALKTHROUGH

9.4.1 Service Identity: The Tag

In most frameworks, you inject a class or a string. In Effect, we use a **Tag**. A Tag is more than a string; it is a type-safe handle that represents both the **Identity** and the **Shape** of a service.

```
| import { Context, Effect } from "effect"
```

```
export interface UserRepo {
  readonly getById: (id: string) => Effect.Effect<User, NotFound>
}

// The Tag is the "Key" in the Environment map
export class UserRepo extends Context.Tag("@app/UserRepo")<
  UserRepo,
  UserRepo
>() {}
```

9.4.2 Pulling from the Environment

When you `yield* UserRepo`, you are telling the runtime: "Stop. Go to the environment map, find the value associated with this Tag, and give it to me."

```
const getUser = (id: string) =>
  Effect.gen(function* () {
    const repo = yield* UserRepo
    return yield* repo.getById(id)
  })
```

9.4.3 Why Tag is Superior

- **No Collisions:** Unique string IDs prevent two different "Logger" services from clashing.
- **Type Safety:** The compiler knows exactly what methods are available on `repo` because they are defined in the Tag.
- **Refactor Friendly:** You can rename the implementation class without changing a single line of business logic that depends on the Tag.

9.5 CHECKPOINT

- Services live in the R channel.
- Use `Context.Tag` to create a unique handle for a service.
- Use `yield*` inside `Effect.gen` to access services.
- Keep requirements static; dynamic data belongs in the function arguments (A).

9.6 EXERCISES

1. Define a Database service with a `saveUser` method.
2. Build a program that yields the Database service and calls `saveUser`.
3. Verify that the program's type includes Database in the requirements channel.

9.7 FOOTGUNS

- **Manual Context Propagation:** Passing service instances as function arguments instead of using the R channel.
- **Leaky Requirements:** Defining service methods that themselves require other services (this should be handled at the Layer level).
- **Concrete Dependency:** Depending on implementation classes directly instead of using Tags.

CANONICAL REFERENCE

- Managing Services

Chapter 10

LAYERS: CONSTRUCTORS FOR SERVICES

10.1 PURPOSE

If services are the "contracts," layers are the "constructors" with lifecycle. This chapter teaches you how to build a robust dependency graph that handles construction logic and resource management outside of your business logic.

10.2 INVARIANTS

- **Construction-Only R:** Dependencies of a service belong in the Layer that constructs it, not in the service's methods.
- **Implicit Memoization:** Within a single dependency graph (provide), each layer instance is instantiated exactly once.
- **Swap-Ready:** Every implementation of a service should be provided by a Layer to facilitate testing and environment switching.

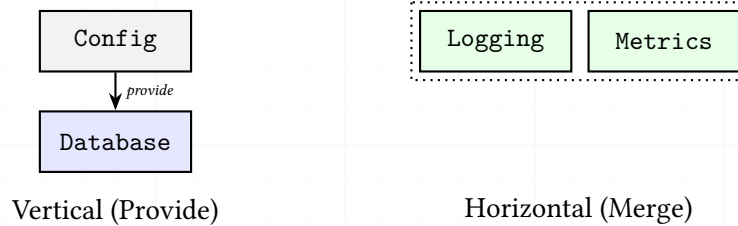
10.3 NEW CONCEPTS

- **Layer:** A recipe for producing a service implementation from optional input dependencies.
- **Constructor:** A function or value that creates an instance of a service.
- **Memoization:** The mechanism that ensures a shared dependency is only initialized once per program execution.

10.4 WALKTHROUGH

10.4.1 Dependency Graph Topology

Building an application environment is like assembling a jigsaw puzzle.



You have two primary ways to combine Layers:

1. **Horizontal Composition** (`Layer.merge`): Combining two independent services into a single "bundle."
2. **Vertical Composition** (`Layer.provide`): Using one service to build another (e.g., using a `Config` to build a `Database`).

```
import { Layer } from "effect"

// Vertical: Database needs Config
const DbLive = Layer.effect(Database,
  Effect.gen(function*() {
    const config = yield* Config
    return { ... }
  })
).pipe(Layer.provide(ConfigLive))

// Horizontal: Bundle everything for the App
const AppLayer = Layer.merge(DbLive, HttpClientLive)
```

10.4.2 Layer Lifecycle

Layers aren't just for injection; they are for **Management**. A layer can be "Scoped," meaning it acquires a resource (like a pool) when the app starts and releases it when the app stops.

```
const PoolLive = Layer.scoped(Pool,
  Effect.acquireRelease(
    createPool(),
    (p) => Effect.sync(() => p.close())
  )
)
```

10.4.3 Sharing and Memoization

Effect is smart. If ten different services in your graph all depend on the same `ConfigLive`, Effect will only initialize that layer **once**. This makes complex dependency graphs highly efficient and predictable.

10.5 CHECKPOINT

- Layers are the implementation recipes for Tags.
- Use `Layer.effect` when a service needs other services to be initialized.

- Swapping layers is the primary mechanism for testing.
- Shared dependencies are memoized automatically by the runtime.

10.6 EXERCISES

1. Build a 4-node dependency graph using only Layers.
2. Introduce a test layer at one specific node in the graph and verify that only that node's implementation changes.

10.7 FOOTGUNS

- **Hidden Dependencies:** Service methods with `R ! =` never force callers to manage context manually.
- **Manual provide:** Calling `Effect.provide` deep inside your logic instead of at the root makes the graph hard to refactor.
- **Tag Collisions:** Using the same string identifier for two different Tags can lead to "missing service" errors at runtime.

CANONICAL REFERENCE

- Managing Layers
- Layer Memoization

Chapter 11

SCOPE: RESOURCES, FINALIZERS, AND "CLEANUP THAT STILL RUNS WHEN EVERYTHING IS ON FIRE"

11.1 PURPOSE

Effect's runtime is explicitly responsible for ensuring resource cleanup. This chapter teaches you how to use **Scope** to manage the lifetime of expensive resources like database connections or file handles, ensuring they are always released even during failures or interruptions.

11.2 INVARIANTS

- **LIFO Finalizers:** Resources are released in the reverse order of their acquisition (Last-In, First-Out).
- **No Leaky Handles:** Resource handles should be managed within Layers; business logic should only interact with the service methods, not the lifecycle of the resource.
- **Interruption-Proof:** Finalizers are guaranteed to run even if the fiber is interrupted during the "use" phase.

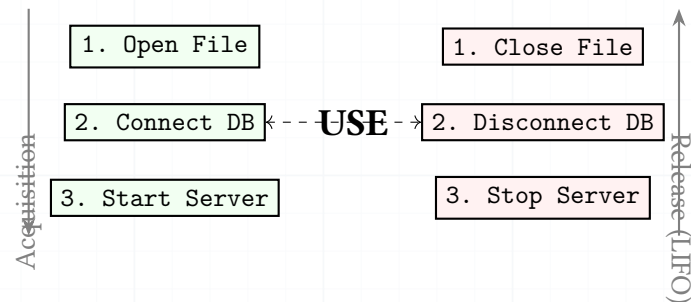
11.3 NEW CONCEPTS

- **Scope:** A handle representing a lifetime boundary. When a scope closes, all registered finalizers run.
- **Finalizer:** A function (`Exit -> Effect`) that runs when a scope is closed.
- **acquireRelease:** The core primitive for safely creating a resource and attaching a finalizer to the current scope.

11.4 WALKTHROUGH

11.4.1 The LIFO Guarantee

The "Last-In, First-Out" (LIFO) order is the most important concept in resource management. It ensures that if a Database depends on a Connection Pool, the Database is closed *before* the Pool.



11.4.2 The canonical primitive: acquireRelease

To manage a resource, you define how to acquire it and how to release it. The result is an Effect that requires a **Scope**.

```
import { Effect } from "effect"

const acquireDb = Effect.log("Connecting to DB...")
const releaseDb = Effect.log("Disconnecting from DB...")

const DbResource = Effect.acquireRelease(acquireDb, () => releaseDb)
// Resulting type: Effect<void, never, Scope>
```

11.4.3 "Just run it safely": Effect.scoped

If you have an Effect that requires a Scope, you must eventually "provide" that scope. `Effect.scoped` is the manual way to do this: it creates a new scope, runs the program, and then closes the scope.

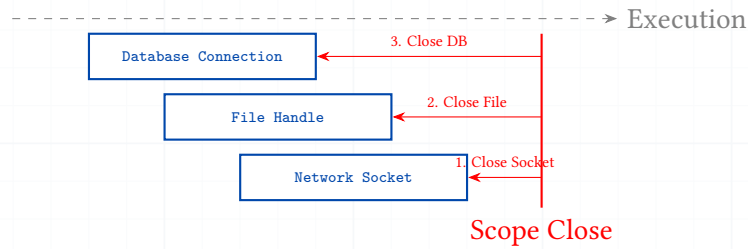
```
const program = Effect.scoped(
  Effect.gen(function* () {
    const db = yield* DbResource
    yield* Effect.log("Using DB...")
  })
)
```

11.4.4 Layers with lifetimes: scoped services

The best place for resource lifetimes is inside a **Layer**. Using `Layer.scoped` allows you to attach a resource's lifetime to the lifetime of the application or a specific test.

11.5 THE ACQUISITION TIMELINE

Resources within a Scope follow a strict LIFO (Last-In, First-Out) release order.



This deterministic cleanup is the "Load-bearing Wall" of resource safety. By ensuring that the last resource acquired is the first one released, Effect prevents the "Zombies" of leaked connections that haunt legacy Promise-based systems.

11.6 CHECKPOINT

- Scope is an explicit lifetime boundary for resources.
- Use `acquireRelease` to ensure finalizers always run.
- `Effect.scoped` handles local lifetimes.
- `Layer.scoped` handles shared service lifetimes.

11.7 EXERCISES

1. Build a `HttpClient` service that owns a keep-alive agent using `acquireRelease`.
2. Write a test layer for that service that needs no `Scope` and verify that the requirements of the program change accordingly.
3. Use `Effect.addFinalizer` to register an extra logging action that runs when a scope closes.

11.8 FOOTGUNS

- **Manual Resource Handles:** Creating a database client via `new` and expecting it to "just work" without an `acquireRelease` wrapper.
- **Scope Leaks:** Forgetting to call `Effect.scoped` on a program that requires a scope, leading to requirements that never get satisfied.
- **Closing Too Early:** Using `Effect.scoped` inside a loop when you actually wanted the resources to live for the duration of the entire loop.

CANONICAL REFERENCE

- Scope
- Resource Management Overview

Chapter 12

RUNTIME AND EXECUTION BOUNDARIES: WHERE THE BLUEPRINT BECOMES REAL- ITY

12.1 PURPOSE

An Effect is just a description—a blueprint of what should happen. The **Runtime** is the engine that interprets that blueprint and turns it into real-world actions. This chapter teaches you how execution boundaries work and how to build reusable runtimes for your applications.

12.2 INVARIANTS

- **Inert Blueprints:** An Effect value never performs any work on its own; it must be passed to a runner (`run*`).
- **Execution Locality:** The runner should reside at the absolute edge of your program (e.g., `main.ts`, React root, or an HTTP route handler).
- **Environment Propagation:** A Runtime carries a specific Context, ensuring that every Effect it runs has access to the same provided services.

12.3 NEW CONCEPTS

- **Runtime:** The configuration and environment used to execute an Effect.
- **ManagedRuntime:** A specialized runtime that manages the lifecycle of its own environment (layers).
- **Execution Boundary:** The point in the code where an Effect is handed off to the runtime for execution.

12.4 WALKTHROUGH

12.4.1 The Interpreter Analogy

Think of an Effect as a script written in a specific language. The **Runtime** is the actor that reads the script and performs the actions. A single script can be “performed” by different runtimes—one that logs to the console for development, and one that logs to an ELK stack for production.

12.4.2 Execution Boundaries

Law 1 and Law 2 mandate that runtimes live at the “Edge.” In a Node.js server, this edge is usually the entry point of your process.

```
import { Effect } from "effect"

const blueprint = Effect.log("Calculating results...")

// THE EDGE: The only place we call run*
Effect.runPromise(blueprint)
```

12.4.3 Reusable Environments: ManagedRuntime

When your application has a complex set of dependencies (Database, Cache, Metrics), you don’t want to rebuild the dependency graph for every request. A `ManagedRuntime` allows you to “bake” the environment once.

```
import { ManagedRuntime, Layer } from "effect"
import { AppLayer } from "../AppLayer"

// Build the runtime once at startup
const runtime = ManagedRuntime.make(AppLayer)

// Use the pre-baked environment to run effects
// This is extremely efficient: services are already initialized.
export const run = runtime.runPromise
```

By using `ManagedRuntime`, you ensure that shared resources like database pools are shared across all executions, following the memoization rules of Law 2.

12.5 CHECKPOINT

- Effects are lazy and inert until run.
- Place your execution boundaries at the edges of your system.
- Use `ManagedRuntime` to manage service lifetimes for long-running processes or CLI tools.
- Avoid calling `run*` inside library or domain logic.

12.6 EXERCISES

1. Build a `ManagedRuntime` from your `AppLayer` and use it to run a simple health-check Effect.

2. Scan a hypothetical "legacy" codebase and replace an internal `Effect.runPromise` call with an exported `Effect` value that is run at the entry point.
3. Experiment with `Effect.runSync` and observe what happens when you pass it an asynchronous `Effect`.

12.7 FOOTGUNS

- **Premature Execution:** Calling `Effect.runSync` deep inside a function just to "get the value," which breaks composability.
- **Multiple Runtimes:** Creating too many `ManagedRuntime` instances, which can lead to redundant resource allocations and conflicting service states.
- **Implicit Global Context:** Depending on the default global runtime for production logic instead of explicitly defining your application's environment requirements.

CANONICAL REFERENCE

- Runtime
- Running Effects

Chapter 13

COMPOSITION ROOTS: ONE DEPENDENCY GRAPH, ONE PROVIDE, ONE RUN

13.1 PURPOSE

A program's architecture is defined by how its dependencies are wired together. This chapter teaches you the principle of the **Composition Root**: the single location in your application where the dependency graph is constructed and the program is executed.

13.2 INVARIANTS

- **Inert Libraries**: Modules outside the entry point should only export Effects and Layers; they must never call `run*` or `provide`.
- **Single AppLayer**: All service implementations should be aggregated into a single `AppLayer` that serves as the blueprint for the entire environment.
- **Unified Run**: There should be exactly one call to a runner (`run*`) per logical program execution.

13.3 NEW CONCEPTS

- **Composition Root**: The entry point where all components are wired together.
- **AppLayer**: The top-level Layer representing the total environment of the application.
- **Wiring**: The process of connecting service interfaces to their specific implementations.

13.4 WALKTHROUGH

13.4.1 Dependency Graph Assembly

Building an `AppLayer` is about deciding how your services relate to each other. We use two primary operators:

1. **Layer.provide:** "Vertical" composition. One layer feeds its output into the input of another.
2. **Layer.merge:** "Horizontal" composition. Multiple layers are bundled together to satisfy a broader requirement.

```
import { Layer } from "effect"
import { DbLive, ConfigLive, ApiLive, LoggerLive } from "../layers"

// 1. The Database needs Configuration
const DbWithConfig = DbLive.pipe(Layer.provide(ConfigLive))

// 2. The API needs the Database AND Logging
const ApiFull = ApiLive.pipe(
  Layer.provide(DbWithConfig),
  Layer.provide(LoggerLive)
)

// 3. Assemble the final Application Blueprint
export const AppLayer = Layer.mergeAll(
  ApiFull,
  LoggerLive // Bundled for observability
)
```

13.4.2 The Main Entry Point

The Composition Root is the only file that imports the "Live" implementations. Every other file in your system should only import the `Context.Tag`.

```
import { NodeRuntime } from "@effect/platform-node"
import { MainProgram } from "../logic"
import { AppLayer } from "../CompositionRoot"

// satisfying all obligations at once
const runnable = MainProgram.pipe(
  Effect.provide(AppLayer)
)

NodeRuntime.runMain(runnable)
```

This separation of concerns ensures that your business logic remains "Implementation Blind," making it trivial to swap your database for a mock in your test suite.

13.5 CHECKPOINT

- Avoid "Layer Fragmentation" by pushing provide to the top.
- Logic modules should be implementation-agnostic.
- Use `Layer.mergeAll` or `Layer.provide` to build the `AppLayer`.
- There should be exactly one call to `run*` per program.

13.6 EXERCISES

1. Refactor a small existing application (or create a mock one with three files) so that only one file calls `provide` and `run`.
2. Experiment with moving a `provide` call deep into a service and observe how it affects your ability to write a unit test for that service.

13.7 FOOTGUNS

- **Layer Leakage:** Importing a "Live" layer into a business logic file instead of just the service Tag.
- **Multiple Main Files:** Having multiple entry points that wire the same dependencies differently, leading to inconsistent behavior in production vs. testing.
- **Hardcoded Providers:** Calling `Effect.provide` inside a service's constructor, which prevents the consumer from swapping its dependencies.

CANONICAL REFERENCE

- Managing Layers
- Running Effects

Chapter 14

CONFIGURATION AS A FIRST-CLASS PROGRAM

14.1 PURPOSE

Configuration in Effect is not a simple lookup of environment variables. It is a first-class program that describes how your application's settings should be loaded and validated. This chapter teaches you how to build declarative, type-safe configuration specs that handle defaults, secrets, and validation errors gracefully.

14.2 INVARIANTS

- **Declarative Spec:** A `Config` value is a description of how to load a value, not the value itself.
- **Early Validation:** Configuration should be loaded and validated during the construction of a `Layer`, preventing the program from running with invalid settings.
- **Redacted by Default:** Sensitive configuration (like API keys) should use `Config.redacted` to prevent accidental logging.

14.3 NEW CONCEPTS

- **Config:** A description of a configuration value and how to retrieve it.
- **Redacted:** A wrapper for sensitive data that hides its value from logs and standard stringification.
- **ConfigProvider:** A service that knows how to resolve `Config` descriptions into actual values (e.g., from environment variables, files, or secret managers).

14.4 WALKTHROUGH

14.4.1 Nested Configuration

Most applications have grouped settings. You can use `Config.all` to map environment variables into nested domain objects.

```
import { Config, Effect } from "effect"

const DatabaseConfig = Config.all({
  host: Config.string("DB_HOST").pipe(Config.withDefault("localhost")),
  port: Config.number("DB_PORT").pipe(Config.withDefault(5432)),
  password: Config.redacted("DB_PASSWORD")
})

const AppConfig = Config.all({
  db: DatabaseConfig,
  apiKey: Config.redacted("API_KEY")
})
```

14.4.2 Loading the Config

Because configuration is a program, you yield it inside a generator. If an environment variable is missing or has the wrong type, the program fails with a detailed `ConfigError`.

```
const program = Effect.gen(function* () {
  const config = yield* AppConfig
  yield* Effect.log(`Connecting to ${config.db.host}`)
})
```

14.4.3 Standardizing on ConfigProviders

Effect allows you to change *where* configuration comes from without changing the `Config` definitions. You can swap the default `ProcessEnv` provider for a `Json` file provider or a remote `Vault` provider at the `Composition Root`.

```
import { ConfigProvider } from "effect"

// Read config from a local file instead of environment variables
const FileProvider = ConfigProvider.fromJson({ ... })

const runnable = program.pipe(
  Effect.withConfigProvider(FileProvider)
)
```

14.5 CHECKPOINT

- Configuration is a first-class program, not a global lookup.
- Use `Config.redacted` for sensitive data.
- Use `Config.withDefault` to ensure your application can start in development environments.
- Validate configuration early, during `Layer` construction.

14.6 EXERCISES

1. Define a nested configuration object for a Database service (host, port, and password).
2. Use `Config.redacted` for the database password and verify that logging the config object does not reveal the secret.
3. Build a program that attempts to load the configuration and prints a clear error message if the PORT is invalid.

14.7 FOOTGUNS

- **Manual `process.env` Access:** Accessing environment variables directly bypassing `Config` makes your code impossible to test without global state mutation.
- **Late Validation:** Reading configuration only when a specific function is called, leading to application crashes minutes or hours after startup.
- **Secret Exposure:** Forgetting to use `Config.redacted` for sensitive data, leading to passwords appearing in centralized log aggregators.

CANONICAL REFERENCE

- Configuration

Chapter 15

SCHEMA-FIRST BOUNDARIES

15.1 PURPOSE

Data integrity is the foundation of a stable system. This chapter teaches you how to use **Schema** to define your data models once and use them for validation, transformation, and serialization across your entire application boundary.

15.2 INVARIANTS

- **Single Source of Truth:** A Schema defines both the TypeScript type and the runtime validation logic.
- **Parse, Don't Validate:** Schema should be used to transform untrusted input into trusted domain objects, not just to check for errors.
- **Opaque Boundary:** No untrusted data (JSON from API, DB rows) should enter the business logic without being processed by a Schema.

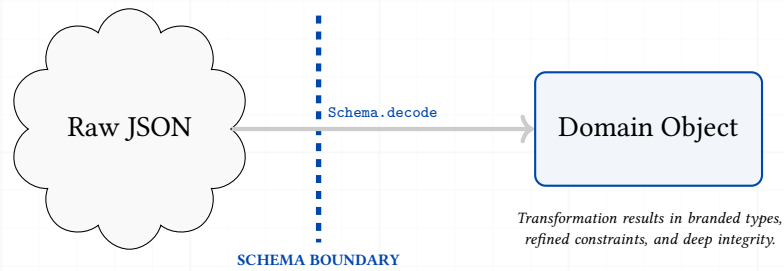
15.3 NEW CONCEPTS

- **Schema:** A bidirectional description of a data structure.
- **Parse:** The process of taking raw input and producing a valid, typed object.
- **Transformation:** A schema that also changes the data shape (e.g., parsing a string into a Date).

15.4 WALKTHROUGH

15.4.1 The Checked Zone

Schema is not just for validation; it is for **Transformation**. It defines the boundary between the "Untrusted World" and your "Pure Domain."



15.4.2 Defining your domain

We use `Schema.Class` to define our core entities.

15.4.3 Beyond Basic Types: Branding

In standard TypeScript, a `UserId` is just a `string`. This is dangerous: you can accidentally pass an `OrderId` where a `UserId` is expected. Effect **Schema** allows you to create “Branded Types” that are enforced at runtime.

```
import { Schema } from "effect"

// Define a unique brand for User ID
export const UserId = Schema.String.pipe(Schema.brand("UserId"))
export type UserId = Schema.Schema.Type<typeof UserId>

const User = Schema.Struct({
  id: UserId,
  name: Schema.String
})
```

15.4.4 The Transformation Pipeline

Schema is not just for validation; it’s for **Materialization**. You can define how a raw string from a database transforms into a sophisticated domain object.

```
// A schema that transforms a string into a Date
const DateFromSelf = Schema.Date

// A schema that transforms a string into a URL object
const UrlFromSelf = Schema.transform(
  Schema.String,
  Schema.instanceOf(URL),
  {
    decode: (s) => new URL(s),
    encode: (u) => u.toString()
  }
)
```

15.4.5 Schema-First Error Handling

When you use `Schema.decodeUnknown`, the result is an `Effect` that can fail with a `ParseError`. This error is highly detailed, telling you exactly which field failed and why (e.g., "Expected number, got string"). This is the foundation of Law 4: Boundary Integrity.

15.5 CHECKPOINT

- Use `Schema` to define all application boundaries.
- Derive types from schemas to avoid duplication.
- Use `decodeUnknown` or `decodeUnknownSync` to ingest external data.
- Leverage `Schema.TaggedError` for type-safe failures.

15.6 EXERCISES

1. Define a schema for a `Product` with an `id`, a `price` (which must be positive), and a list of `tags`.
2. Build a transformation schema that takes a string representing an ISO date and produces a JavaScript `Date` object.
3. Use `Schema.parseJson` to decode a mock response from an API and handle the error case where the JSON is malformed.

15.7 FOOTGUNS

- **Manual JSON Parsing:** Using `JSON.parse` directly and then casting with `as` bypasses all the runtime guarantees of the `Schema`.
- **Too Generic Schemas:** Using `Schema.Any` or `Schema.Unknown` too often, which effectively turns off the type-safety you are trying to achieve.
- **Validation at the Center:** Delaying parsing until deep inside your business logic, which forces every function to handle potential data inconsistencies.

CANONICAL REFERENCE

- Schema Introduction

Chapter 16

ONE END-TO-END SLICE

16.1 PURPOSE

Architecture is often theoretical until you see a complete slice of a system. This chapter pulls together all the principles—Services, Layers, Schemas, and Runtimes—to build a single, robust, end-to-end feature. You will learn how to structure a program from the incoming request to the final execution.

16.2 INVARIANTS

- **Layered Construction:** Dependencies are wired in the **Layer** graph, never manually passed to service methods.
- **Boundary Integrity:** Every external input is parsed using **Schema** into a trusted domain model.
- **Managed Execution:** The application's entry point uses `NodeRuntime.runMain` to ensure graceful shutdowns and proper finalization.

16.3 NEW CONCEPTS

- **Slice:** A complete vertical implementation of a feature, from API boundary to database.
- **Feature:** A unit of business value composed of multiple Effect programs and services.
- **Orchestration:** The logic that coordinates multiple services within a single Effect program.

16.4 WALKTHROUGH

16.4.1 Building a Vertical Slice

Let's assemble a complete feature for a User Creation system.

1. The Contract (Repository)

We define the service Tag and its interface.

```
export interface UserRepo {
  readonly create: (name: string) => Effect<User, DbError>
}
export class UserRepo extends Context.Tag("UserRepo")<UserRepo,
  UserRepo>() {}
```

2. The Logic (Service)

The logic depends only on the UserRepo Tag.

```
const createUser = (input: unknown) => Effect.gen(function*() {
  // Decode the boundary
  const { name } = yield* Schema.decodeUnknown(UserInput)(input)
  const repo = yield* UserRepo
  return yield* repo.create(name)
})
```

3. The Implementation (Layer)

This is where the database-specific code lives.

```
const UserRepoLive = Layer.effect(UserRepo, Effect.gen(function*() {
  const db = yield* SqlClient // Needs a separate database client
  return {
    create: (name) => db.run(`INSERT INTO users...`)
  }
}))
```

4. The Assembly (Composition Root)

Finally, we wire them together and run.

```
const program = createUser({ name: "Agusti" }).pipe(
  Effect.provide(UserRepoLive),
  Effect.provide(SqlClientLive) // satisfies UserRepo's dependency
)

NodeRuntime.runMain(program)
```

This horizontal wiring (step 4) keeps the implementation details entirely separate from the business value (step 2).

16.5 CHECKPOINT

- Features should be modeled as vertical slices.
- Dependencies are handled by the **Layer** graph.
- External data must be parsed into trusted domain models.

- Use `NodeRuntime.runMain` for clean application entry points.

16.6 EXERCISES

1. Build a complete "Task Manager" slice: define a `TaskRepo` service, a **Layer** that implements it with an in-memory array, a **Schema** for task input, and a `runMain` entry point that creates a task.
2. Add a validation rule to the task schema (e.g., description cannot be empty) and observe how the program fails if invalid input is provided.

16.7 FOOTGUNS

- **Manually Passing Context:** Breaking the R channel by passing service instances through deep function hierarchies manually.
- **Incomplete Error Mapping:** Leaving raw platform errors (like `FetchError`) in the Success channel instead of mapping them to domain errors.
- **Bypassing runMain:** Using `Effect.runPromise` for the main entry point, which can cause the process to hang or leak resources if interrupted.

CANONICAL REFERENCE

- Platform Runtime

Chapter 17

THE "RULES OF THE MONOREPO"

17.1 PURPOSE

Scaling Effect beyond a single project requires a disciplined approach to code organization. This chapter teaches you the "Rules of the Monorepo"—how to structure packages, enforce execution boundaries across boundaries, and maintain a clean, navigable codebase as your team and system grow.

17.2 INVARIANTS

- **Outermost Run:** Execution (`run*`) must be strictly confined to the application entry points (leaf packages).
- **Dependency Inversion:** Internal packages must depend only on service Tags, never on concrete "Live" layers from other packages.
- **Top-Level Provide:** The wiring of implementations to interfaces happens exclusively at the Composition Root of the final application package.

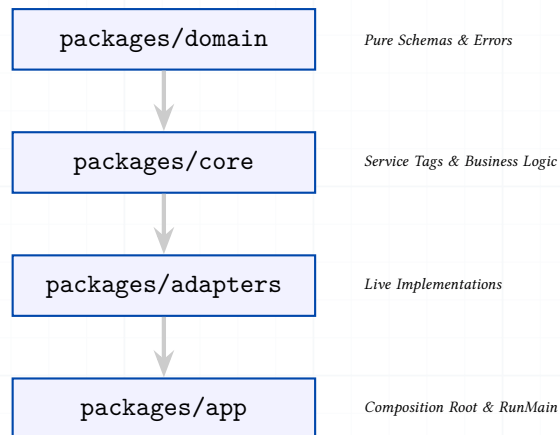
17.3 NEW CONCEPTS

- **Internal Package:** A package containing pure business logic and service interface definitions (Tags).
- **Adapter Package:** A package containing concrete implementations of services for a specific platform (e.g., AWS, Node.js).
- **App Package:** The final "shell" package that wires everything together and runs the application.

17.4 WALKTHROUGH

17.4.1 Dependency Tier Topology

Large systems rot when dependencies become circular. Effect monorepos solve this by enforcing a strict vertical flow.



17.4.2 The non-negotiables

When code is spread across multiple packages...

17.4.3 The strict hierarchy of packages

A successful Effect monorepo is built on a "One-Way Dependency" rule. This prevents circular dependencies and architectural rot.

1. **domain**: Pure schemas and errors. **Depends on nothing.**
2. **core**: Service Tags and business logic. **Depends on domain.**
3. **adapters**: Platform implementations (Postgres, AWS). **Depends on core.**
4. **app**: Composition Root and entry point. **Depends on core and adapters.**

17.4.4 Export Rules: What to show

- **Core packages** must NEVER export a "Live" layer. They only export Tags and Logic.
- **Adapter packages** should export exactly one scoped Layer.
- **Domain packages** should only export `Data.Class` and `Schema`.

17.4.5 Review Checklist for PRs

- **Rule of 3**: Does a shared library call `runPromise`? (REJECT)
- **Implementation Leak**: Does a core file import from an adapter? (REJECT)
- **Lazy wiring**: Is the `AppLayer` the only place where `provide` is called? (APPROVE)

By enforcing these boundaries, you ensure that your system remains navigable even when it grows to hundreds of thousands of lines of code.

17.5 CHECKPOINT

- Enforce strict execution boundaries.
- Decouple logic from platform-specific implementations.
- Centralize all dependency wiring in the final application package.
- Use package-level partitioning to prevent architectural drift.

17.6 EXERCISES

1. Draw a package dependency graph for a "Weather App" that has a web frontend and a Node.js backend, showing which packages are shared and which are platform-specific.
2. Perform a "Mock Review": find three architectural violations in a hypothetical PR where a developer has added a database connection call directly inside a shared "domain" package.

17.7 FOOTGUNS

- **Circular Dependencies:** Attempting to import from the "app" package back into a "core" library.
- **Layer Leakage:** Publishing a library that includes "Live" layers, which forces consumers to adopt your specific implementation choices (like a specific database driver).
- **Boundary Erasure:** Using any or unknown when passing data between packages, which destroys the type-safe contract established by your schemas.

CANONICAL REFERENCE

- Code Style Guidelines

Tier IV

CONCURRENCY & RESILIENCE



Blueprint Overview

CONTENTS

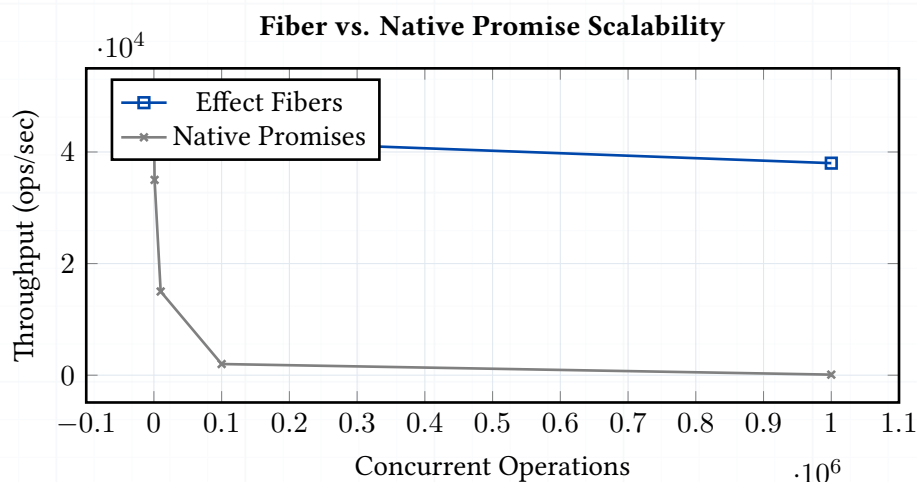
18 Fibers: structured concurrency, cancellation, and not lying about parallelism	81
18.1 The Throughput Engine	81
18.2 Purpose	81
18.3 Invariants	82
18.4 New concepts	82
18.5 Walkthrough	82
18.5.1 The Chain of Command	82
18.5.2 Automatic Interruption	83
18.5.3 When to use forkDaemon	83
18.5.4 Manual Control: Fiber.interrupt	83
18.6 The Runtime Loop	83
18.7 Checkpoint	84
18.8 Exercises	84
18.9 Footguns	84
19 Resilience: retries, timeouts, and schedules as policy	85
19.1 Purpose	85
19.2 Invariants	85
19.3 New concepts	85
19.4 Walkthrough	85
19.4.1 Policy Algebra: Composable Resilience	85
19.4.2 Timeout as an Operational State	86
19.5 Checkpoint	86
19.6 Exercises	87
19.7 Footguns	87
20 Advanced Concurrency Patterns	89
20.1 Purpose	89
20.2 Concurrency Archetypes	89
21 Coordination primitives: building deterministic concurrency graphs	91
21.1 Purpose	91
21.2 Invariants	91
21.3 New concepts	91
21.4 Walkthrough	91
21.4.1 Backpressure as a Safety Valve	92
21.4.2 When to use PubSub	92
21.4.3 Semaphores: Protecting Fragile Resources	92
21.5 Checkpoint	92

Chapter 18

FIBERS: STRUCTURED CONCURRENCY, CANCELLATION, AND NOT LYING ABOUT PARALLELISM

18.1 THE THROUGHPUT ENGINE

Architecture by Construction is not just about safety; it is about extreme efficiency.



Notice the "Cliff of Collapse" for native promises as concurrency approaches 10^4 . This collapse is caused by the overhead of OS thread management and unmanaged microtask queues. Effect Fibers maintain near-linear performance because they are logical, managed by a deterministic loop.

18.2 PURPOSE

Effect uses a fiber-based concurrency model. This chapter teaches you how to manage concurrent tasks safely using **Fibers**, following the principles of structured concurrency where child lifetimes are explicitly tied to their parents.

18.3 INVARIANTS

- **Parent-Child Hierarchy:** By default, child fibers are interrupted when their parent fiber finishes or is interrupted.
- **Explicit Join:** You must explicitly join a fiber to receive its success value or error.
- **Safe Interruption:** Interruption is not a "kill -9"; it's a signal that causes the target fiber to run its finalizers and then terminate.

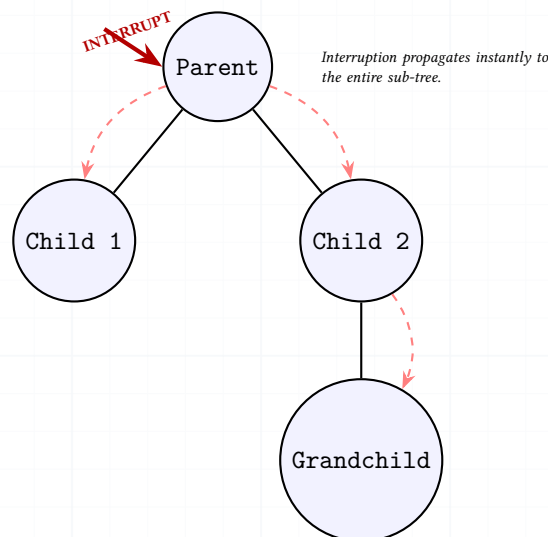
18.4 NEW CONCEPTS

- **Fiber:** A lightweight virtual thread managed by the Effect runtime.
- **Forking:** Starting an Effect on a new fiber in the background.
- **Joining:** Waiting for a fiber to complete and retrieving its result.
- **Interruption:** Stopping a running fiber and ensuring its finalizers run.

18.5 WALKTHROUGH

18.5.1 The Chain of Command

In Effect, fibers are not independent "fire and forget" threads. They exist in a strict hierarchy.



If Fiber A forks Fiber B, Fiber A is the **Supervisor**.

```

import { Effect } from "effect"

const parent = Effect.gen(function* () {
  yield* Effect.log("Parent starting...")

  // Start a child in the background
  const child = yield* Effect.log("Child working...").pipe(
    Effect.delay("10 seconds"),
  
```

```

    Effect.fork
  )

  yield* Effect.sleep("1␣second")
  return "Parent␣done"
})

```

18.5.2 Automatic Interruption

The most powerful guarantee of Law 6 is that when `parent` finishes (or fails), `child` is **automatically interrupted**. You don't need to manually keep track of handles. `Effect` handles the "garbage collection of execution" for you.

18.5.3 When to use `forkDaemon`

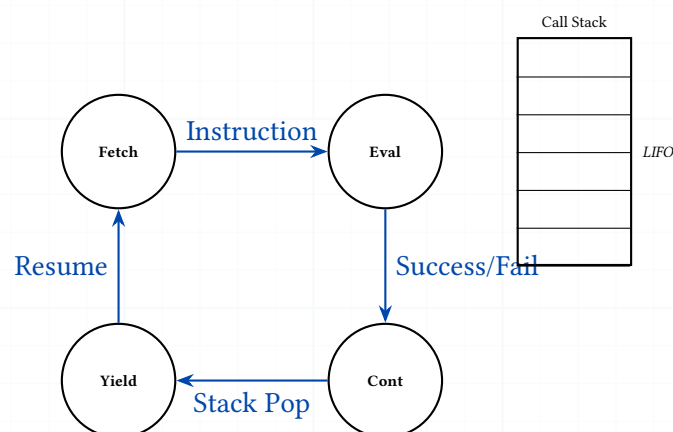
Sometimes you truly want a task to outlive the request (e.g., writing to a metric store or an audit log). In these rare cases, use `Effect.forkDaemon`. This attaches the fiber to the **Global Scope**, meaning it will only be interrupted when the entire application shuts down.

18.5.4 Manual Control: `Fiber.interrupt`

If you need to stop a fiber before its parent finishes, you can call `interrupt`. This is not a "kill" signal; it is a request for the fiber to gracefully shut down, run its finalizers (from **Scope**), and exit.

18.6 THE RUNTIME LOOP

Every Fiber is a state machine processed by the runtime.



*The runtime is essentially a **while loop**. It "steps" through the instructions of your `Effect` until it hits an `async` boundary or completes. This is why Fiber-based concurrency is so efficient: it avoids OS thread context switching.*

18.7 CHECKPOINT

- Fibers are the unit of concurrency in Effect.
- Effect implements structured concurrency by default.
- `Fiber.join` allows you to await a background task.
- Interruption is safe and always runs finalizers.

18.8 EXERCISES

1. Write a program that forks two children; interrupt the parent; observe that both children terminate automatically.
2. Rewrite the same program using `forkDaemon` and observe that the children continue to run after the parent finishes.
3. Use `Effect.race` to run two fibers and take only the result of the first one to finish.

18.9 FOOTGUNS

- **Orphan Fibers:** Using `Effect.runFork` deep in your logic creates a fiber with no supervisor, making it impossible to stop gracefully.
- **Unjoined Forks:** Forgetting to join a fiber means you may never know if it failed.
- **Interruption Deafness:** Wrapping a blocking synchronous operation that does not check for interruption can prevent a fiber from stopping when requested.

CANONICAL REFERENCE

- Fibers
- Basic Concurrency

Chapter 19

RESILIENCE: RETRIES, TIMEOUTS, AND SCHEDULES AS POLICY

19.1 PURPOSE

In ordinary JS systems, "retry" and "timeout" are typically inconsistently applied or require complex wrappers. Effect treats these as first-class operators that can be applied to any Effect, making it easy to build systems that handle failure gracefully and predictably.

19.2 INVARIANTS

- **Policy Composition:** Resilience policies (like retry schedules) should be defined separately from business logic and composed using pipe.
- **Timeout is a State:** A timeout is an expected operational result, not a mysterious "uncaught exception."
- **Deterministic Retries:** Retries should be driven by a **Schedule**, allowing for precise control over intervals and repetition.

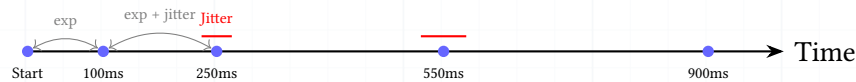
19.3 NEW CONCEPTS

- **Schedule:** A description of a recurring pattern of time, used to control retries and repetition.
- **Retry:** Re-executing an Effect upon failure based on a schedule.
- **Timeout:** Forcing an Effect to fail if it does not complete within a specified duration.

19.4 WALKTHROUGH

19.4.1 Policy Algebra: Composable Resilience

In most systems, retry logic is a mess. In Effect, a resilience policy is a mathematical pattern of time.



Exponential Backoff + Jitter: Protecting your downstream services.

In most systems, retry logic is a mess of `setTimeout` and counters.

```
import { Effect, Schedule } from "effect"

// 1. Start with exponential backoff
const base = Schedule.exponential("100_millis")

// 2. Add "Jitter" to prevent thundering herd
const jittered = Schedule.jittered(base)

// 3. Cap the maximum delay at 3 seconds
const capped = Schedule.intersect(
  jittered,
  Schedule.spaced("3_seconds")
)

// 4. Limit to 10 total retries
const policy = Schedule.intersect(capped, Schedule.recurs(10))

const resilientCall = apiCall.pipe(Effect.retry(policy))
```

19.4.2 Timeout as an Operational State

Law 3 tells us that timeouts shouldn't just "throw." Use `Effect.timeout` to move time-based failure into the type system.

```
const program = resilientCall.pipe(
  Effect.timeout("30_seconds"),
  Effect.catchTag("TimeoutException", () =>
    Effect.logError("Service_unavailable_-_timed_out")
  )
)
```

This approach ensures that your application never "hangs" on a zombie connection. If the clock hits 30 seconds, the fiber is interrupted, resources are cleaned up, and the error is handled.

19.5 CHECKPOINT

- Resilience policies are decoupled from logic.
- `Schedule` allows for complex retry logic (backoff, jitter, recursion).
- Use `Effect.timeout` to prevent hanging fibers.
- Combine policies using `pipe` or `Schedule.intersect`.

19.6 EXERCISES

1. Wrap an external API call with an exponential backoff retry policy and a 5-second timeout.
2. Define a "standard" schedule policy once and apply it to three different operations in a single program.
3. Use `Effect.onEveryRetry` to log a warning message each time an operation is retried.

19.7 FOOTGUNS

- **Infinite Retries:** Forgetting to cap a `Schedule.exponential` or similar policy, leading to fibers that retry forever.
- **Opaque Timeouts:** Not handling the `TimeoutException` explicitly, making it hard to debug why an operation stopped.
- **Resource Starvation:** Retrying expensive database operations too aggressively without jitter, which can cause "thundering herd" problems.

CANONICAL REFERENCE

- Retrying
- Timing Out

Chapter 20

ADVANCED CONCURRENCY PATTERNS

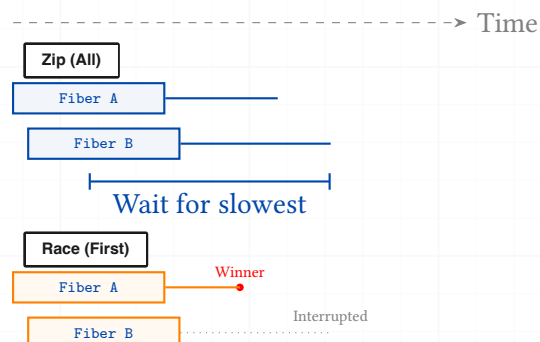
20.1 PURPOSE

Advanced concurrency requires more than just fibers; it requires high-level primitives for signaling and synchronization. This chapter explores coordination patterns like handshakes and deferred results that allow you to build sophisticated parallel workflows.

CANONICAL DOCUMENTATION

20.2 CONCURRENCY ARCHETYPES

Effect provides different ways to combine concurrent operations.



Execution Capture: `core.concurrency.deferred-handshake`

Deferred handshake **Deferred** lets one fiber publish a result exactly once while other fibers wait without blocking a thread.

Status: Stable

Prereqs: Fibers, Error model

Last verified: 2026-01-30

MINIMAL MODEL

- A **Deferred**<A,E> starts empty.
- A writer completes it with **Deferred.succeed** or **Deferred.fail**.
- Readers use **Deferred.await**, suspending the fiber until completion.

CANONICAL PATTERN

```
import { Deferred, Effect } from "effect"

/**
 * A structured "signal" from one fiber to another.
 *
 * - one writer completes the Deferred
 * - one or more readers await it
 */
export const deferredHandshake = Effect.gen(function* () {
  const ready = yield* Deferred.make<string, never>()

  const worker = Effect.gen(function* () {
    // pretend we did some work
    yield* Effect.sleep("50_millis")
    yield* Deferred.succeed(ready, "ready")
  })

  yield* Effect.fork(worker)

  // waits without blocking a thread
  const msg = yield* Deferred.await(ready)
  return msg
})
```

TESTS

This entry has an executable test in `packages/bible-examples/test/deferred.test.ts`. **EXERCISES**

1. Create a producer fiber that completes a **Deferred** after a delay, and multiple consumer fibers that await it.
2. Implement a timeout using **Effect.race** between awaiting the deferred and a sleep effect.

CANONICAL REFERENCE

- `Deferred`

Chapter 21

COORDINATION PRIMITIVES: BUILDING DETERMINISTIC CONCURRENCY GRAPHS

21.1 PURPOSE

Coordination is the art of synchronizing multiple fibers without resorting to shared mutable state or complex locks. This chapter teaches you how to use primitives like **Deferred** and **Latch** to build deterministic concurrency graphs where fibers wait for each other in a structured, safe manner.

21.2 INVARIANTS

- **Single Assignment:** A **Deferred** can be completed exactly once; subsequent attempts are ignored.
- **No Thread Blocking:** Awaiting a coordination primitive suspends the fiber without blocking the underlying JavaScript execution thread.
- **Atomic State Transitions:** Primitives like **Semaphore** and **Latch** handle state updates atomically, ensuring correctness in highly concurrent environments.

21.3 NEW CONCEPTS

- **Deferred:** A variable that starts empty and can be completed once with a success or failure.
- **Latch:** A synchronization primitive that blocks fibers until a specific signal is received.
- **Semaphore:** A tool for limiting the number of fibers that can access a resource concurrently.

21.4 WALKTHROUGH

21.4.1 Backpressure as a Safety Valve

In many applications, a fast producer (e.g., an HTTP server) can overwhelm a slow consumer (e.g., a PDF generator), leading to a crash. Effect's Queue provides "Implicit Backpressure" to solve this.

```
import { Queue, Effect, Fiber } from "effect"

const flow = Effect.gen(function* () {
  // 1. Create a queue that only holds 2 tasks
  const queue = yield* Queue.bounded<string>(2)

  // 2. Slow consumer (takes 1 second per task)
  const consumer = Queue.take(queue).pipe(
    Effect.flatMap((task) => Effect.log(`Processing_${task}`)),
    Effect.delay("1_second"),
    Effect.forever,
    Effect.fork
  )

  // 3. Fast producer
  yield* Queue.offer(queue, "Task_1")
  yield* Queue.offer(queue, "Task_2")
  yield* Effect.log("Queue_is_full._Next_offer_will_SUSPEND.")

  yield* Queue.offer(queue, "Task_3") // This line waits automatically!
  yield* Effect.log("Task_3_finally_offered.")
})
```

21.4.2 When to use PubSub

While a Queue ensures a task is handled by **one** consumer, a PubSub (Publish-Subscribe) allows a task to be handled by **every** subscriber. This is ideal for notification systems or real-time event buses.

21.4.3 Semaphores: Protecting Fragile Resources

If you have a legacy API that can only handle 5 requests at a time, wrap the call in a **Semaphore**.

```
const rateLimit = yield* Effect.makeSemaphore(5)
const limitedApi = rateLimit.withPermits(1)(apiCall)
```

This is the ultimate expression of control: you are no longer at the mercy of the environment; you are defining the environment's rules.

21.5 CHECKPOINT

- Coordination primitives allow fibers to synchronize safely.
- Deferred is for one-to-many communication.
- Semaphore limits concurrent access.

- Use the concurrency option to prevent resource exhaustion.

21.6 EXERCISES

1. Implement a "singleflight" pattern using `Deferred`: if multiple fibers request the same key, only one should perform the work while others await.
2. Build a "readiness gate" for an application: no incoming requests should be processed until a specific initialization service signals it is ready.
3. Use a `Semaphore` to ensure that only one fiber at a time can write to a specific log file.

21.7 FOOTGUNS

- **Deadlocks**: Creating a circular dependency where Fiber A awaits Fiber B, and Fiber B awaits Fiber A.
- **Unbounded Concurrency**: Forgetting to pass the concurrency option when mapping over a large array, potentially crashing the process.
- **Awaiting Indefinitely**: Awaiting a `Deferred` that will never be completed due to an unhandled failure in the producer fiber.

CANONICAL REFERENCE

- `Deferred`
- `Latch`

Chapter 22

STREAMS: "EFFECT, BUT MANY VALUES"

22.1 PURPOSE

Standard asynchronous iterators in JavaScript lack high-quality resource management and error handling. **Stream** is "Effect, but for many values." This chapter teaches you how to build robust, back-pressured data pipelines that handle I/O, sockets, and complex transformations safely.

22.2 INVARIANTS

- **Lazy Descriptions:** A `Stream` value performs no work until it is explicitly consumed (run).
- **Implicit Backpressure:** Consumption of a stream is "pull-based," ensuring that the producer does not overwhelm the consumer.
- **Managed Lifetime:** Any resources acquired by a stream (file handles, sockets) are guaranteed to be released when the stream completes or is interrupted.

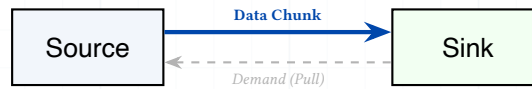
22.3 NEW CONCEPTS

- **Stream:** A lazy, asynchronous program that can emit zero or more values of type `A`.
- **Sink:** A description of how to consume and aggregate the values emitted by a stream.
- **Backpressure:** The mechanism that coordinates the speeds of the producer and consumer to prevent memory exhaustion.

22.4 WALKTHROUGH

22.4.1 The Pull Model: A Stream Vacuum

Most streams (Node.js, RxJS) are "Push" based. Data is pushed at you, and you must scramble to handle it. Effect streams are "Pull" based.



*The Sink creates a "Vacuum."
Data only moves when requested.*

22.4.2 The canonical pipeline

Building a stream is about defining the source and the transformations.

22.4.3 Memory-Efficient I/O

Use streams to process files that are larger than your available RAM. Because Effect pulls data in chunks, your memory usage remains constant regardless of file size.

```
import { Stream, FileSystem } from "@effect/platform"

const processLogs = Effect.gen(function* () {
  const fs = yield* FileSystem.FileSystem

  // 1. Create a stream of chunks from a file
  const stream = fs.stream("access.log")

  // 2. Transform the stream line-by-line
  const lines = stream.pipe(
    Stream.decodeUtf8,
    Stream.splitLines,
    Stream.filter((line) => line.includes("ERROR")),
    Stream.tap((error) => Effect.logError(error))
  )

  // 3. Run the stream
  yield* Stream.runDrain(lines)
})
```

22.4.4 Resource Safety

Even if the stream is interrupted halfway through (e.g., the consumer fiber dies), the `FileSystem` handle is guaranteed to be closed. This is because `Stream` uses **Scope** internally to manage its own lifecycle.

22.5 CHECKPOINT

- Use `Stream` for sequences of values over time.
- Streams handle backpressure and resource management automatically.
- Transform streams using familiar operators (`map`, `filter`, `tap`).
- Execute a stream by calling a consumption operator like `runCollect` or `runForEach`.

22.6 EXERCISES

1. Build a stream that emits a message every second for 5 seconds and then terminates.
2. Implement a data pipeline that reads from an array of numbers, filters the even numbers, and calculates their sum using a **Sink**.
3. Use `Stream.acquireRelease` to build a stream that reads lines from a file and ensures the file is closed even if the stream is interrupted.

22.7 FOOTGUNS

- **Eager Consumption:** Converting a stream to a `Promise<Array>` too early, which defeats the purpose of backpressure and consumes excessive memory for large datasets.
- **Manual Iterator Management:** Attempting to use `for await...of` on a stream instead of using the provided `run*` operators, which can bypass Effect's resource management guarantees.
- **Ignoring Backpressure:** Building streams that produce data much faster than they can be consumed without specifying how to handle the overflow (e.g., using `Stream.buffer`).

CANONICAL REFERENCE

- Stream Introduction
- Creating Streams
- Resourceful Streams

Chapter 23

STATE + BACKPRESSURE PRIMITIVES

23.1 PURPOSE

Concurrent applications often need to share state and manage the flow of data between components. This chapter teaches you how to use primitives like **Ref** and **Queue** to manage shared state safely and build back-pressured communication channels without resorting to global variables or unsafe locks.

23.2 INVARIANTS

- **Atomic Updates:** State updates in a **Ref** are always atomic and consistent, even under heavy concurrency.
- **Implicit Backpressure:** Queue operations like `offer` can suspend the producer if the queue is full, preventing memory exhaustion.
- **Scoped State:** Shared state should be created and managed within the **Layer** graph, ensuring it has a well-defined lifetime.

23.3 NEW CONCEPTS

- **Ref:** A mutable reference to a value that provides atomic updates.
- **Queue:** An asynchronous, back-pressured buffer for communication between producers and consumers.
- **SynchronizedRef:** A specialized **Ref** that allows atomic updates involving effectful operations.

23.4 WALKTHROUGH

23.4.1 Ref: Controlled Shared State

Use **Ref** for simple, non-effectful state updates. Updates are atomic and consistent across fibers.

23.4.2 The Race Condition Hazard

If your state update logic itself requires an Effect (e.g., checking a database before updating a local counter), a simple Ref is not enough. You might suffer from a "Read-Modify-Write" race condition.

23.4.3 SynchronizedRef: Atomic Effectful Updates

SynchronizedRef guarantees that even if the update logic is asynchronous and complex, only one fiber can perform the update at a time.

```
import { SynchronizedRef, Effect } from "effect"

const counter = yield* SynchronizedRef.make(0)

const safeUpdate = SynchronizedRef.updateEffect(counter, (n) =>
  Effect.gen(function* () {
    const isAllowed = yield* checkPermissions() // async check
    return isAllowed ? n + 1 : n
  })
)
```

23.4.4 State Lifecycle

Shared state should almost never be a global variable. Instead, create it inside a **Layer**. This ensures the state is tied to the application's lifecycle and makes it easy to reset for every test case.

```
const CounterLive = Layer.effect(Counter,
  Effect.gen(function* () {
    const ref = yield* Ref.make(0)
    return {
      inc: () => Ref.update(ref, (n) => n + 1),
      get: () => Ref.get(ref)
    }
  })
)
```

23.5 CHECKPOINT

- Use Ref for simple, atomic shared state.
- Use SynchronizedRef if the update logic is effectful.
- Use Queue to coordinate communication between fibers.
- Prefer bounded queues to prevent memory leaks in producers.

23.6 EXERCISES

1. Implement an in-memory rate limiter using a Ref that tracks the number of requests in the current time window.

2. Build a simple "event bus" using a **PubSub** (a specialized `Queue`) where multiple consumers can subscribe to messages.
3. Use `Queue.sliding` to build a logger that drops the oldest messages if the log processing fiber is too slow.

23.7 FOOTGUNS

- **Global Mutable Variables:** Using `let` outside of an `Effect` program instead of a `Ref` inside the context makes your code thread-unsafe and impossible to test reliably.
- **Unbounded Queues:** Using `Queue.unbounded` for high-frequency events can cause your application to crash with an Out of Memory error if the consumer slows down.
- **Deadlocks with SynchronizedRef:** Performing a complex effect inside an update that itself tries to update the same `Ref` can lead to a fiber being suspended forever.

CANONICAL REFERENCE

- `Ref`
- `SynchronizedRef`
- `SubscriptionRef`
- `Queue`

Tier V

OBSERVABILITY & VERIFICATION



Blueprint Overview

CONTENTS

24 Observability: logging and tracing as part of the program's semantics	107
24.1 Purpose	107
24.2 Invariants	107
24.3 New concepts	107
24.4 Walkthrough	107
24.4.1 Contextual Inheritance	108
24.4.2 Spans: The "Navigation Map"	108
24.5 Checkpoint	108
24.6 Exercises	108
24.7 Footguns	109
25 Observability as semantics	111
25.1 Purpose	111
25.2 Invariants	111
25.3 New concepts	111
25.4 Walkthrough	111
25.4.1 Metrics: answering "How Much"	112
25.4.2 Latency Histograms	112
25.4.3 Zero-Overhead Semantic Observability	112
25.5 Checkpoint	112
25.6 Exercises	112
25.7 Footguns	113
26 Effect Platform: the "edge algebra"	115
26.1 Purpose	115
26.2 Invariants	115
26.3 New concepts	115
26.4 Walkthrough	115
26.4.1 Subprocess Pipelines	115
26.4.2 Controlled Execution	116
26.4.3 Interruption Propagation	116
26.5 Checkpoint	116
26.6 Exercises	116
26.7 Footguns	116
27 Platform Runtime	119
27.1 Purpose	119
27.2 Invariants	119
27.3 New concepts	119

Chapter 24

OBSERVABILITY: LOGGING AND TRACING AS PART OF THE PROGRAM'S SEMANTICS

24.1 PURPOSE

Logging and tracing are not secondary concerns in Effect; they are part of the program's semantics. This chapter teaches you how to build "Observable Systems" by leveraging Effect's built-in support for structured logging and distributed tracing.

24.2 INVARIANTS

- **Contextual Hierarchy:** Spans must follow the fiber hierarchy; child fibers inherit the span context of their parent.
- **Semantic Logging:** Log messages should be accompanied by structured annotations (context) rather than raw string interpolation.
- **Zero-Overhead when Disabled:** Tracing and metric collection should have negligible performance impact when no exporters are provided.

24.3 NEW CONCEPTS

- **Span:** A unit of work with a start and end time, potentially nested within a trace.
- **Annotation:** Key-value pairs attached to a fiber context, automatically included in logs.
- **Logger:** A service that processes log events and handles their output (stdout, external services).

24.4 WALKTHROUGH

24.4.1 Contextual Inheritance

Logging in Effect is "Fiber Aware." When you annotate an Effect with a key-value pair, every single log message produced by that Effect—or any child fiber it forks—will automatically carry that data.

```
import { Effect } from "effect"

const processOrder = (orderId: string) =>
  Effect.gen(function*() {
    yield* Effect.log("Validating␣items...")
    yield* Effect.log("Charging␣card...")
  }).pipe(
    Effect.annotateLogs("orderId", orderId)
  )

const main = Effect.gen(function*() {
  yield* processOrder("ORDER-123")
  yield* processOrder("ORDER-456")
})
```

In your centralized logging system (Datadog, Grafana), you can now filter by `orderId` and see the interleaved logs of multiple concurrent orders separated with absolute clarity.

24.4.2 Spans: The "Navigation Map"

Use `Effect.withSpan` to group logs into logical units. In a production trace, this creates a hierarchy that tells you exactly where time is being spent and which branch of your code failed.

```
const program = dbQuery.pipe(
  Effect.withSpan("db.execute", {
    attributes: { table: "users" }
  })
)
```

By following this pattern, your code describes not just what it does, but *how it should be observed*. This is the essence of Law 7.

24.5 CHECKPOINT

- Logging is structured and inherits fiber context.
- Use `annotateLogs` to add shared metadata to all nested logs.
- Use `withSpan` to build a trace of your program's execution.
- Failure context (Cause) is automatically included in error logs.

24.6 EXERCISES

1. Add a root span around a workflow boundary and observe the resulting trace structure.

2. Ensure failures are captured at the span that owns the boundary by deliberately throwing a defect.
3. Use `Effect.logInfo` with three different annotations and verify they appear in the terminal output.

24.7 FOOTGUNS

- **String Interpolation:** Putting variable data directly in the log message instead of using `annotateLogs` makes searching/filtering harder.
- **Too Many Spans:** Creating spans for very small, high-frequency operations can overwhelm your tracing backend.
- **Missing Exporter:** Capturing spans without providing a Tracer layer (like OTel) means your traces are simply discarded by the runtime.

CANONICAL REFERENCE

- Logging
- Tracing

Chapter 25

OBSERVABILITY AS SEMANTICS

25.1 PURPOSE

This chapter explores the deeper philosophy of observability in Effect: it is not a "side output" but a core part of the program's semantics. You will learn how to move beyond simple print statements and build systems where logging, metrics, and tracing are woven into the very fabric of your business logic.

25.2 INVARIANTS

- **Structured Context:** All observability data (logs, traces) must inherit the structured context of the active fiber.
- **Metric Atomicity:** Metrics should be updated as part of the program's logical flow, ensuring they accurately reflect the system's state even during failures.
- **Trace-Driven Rooting:** Every request or background workflow must originate from a well-defined root span.

25.3 NEW CONCEPTS

- **Semantic Logging:** Logging where the focus is on providing structured data for downstream analysis rather than human-readable prose.
- **Metric:** A numerical measurement of system behavior over time (Counter, Gauge, Histogram).
- **Distributed Tracing:** The tracking of a single request or workflow as it moves across multiple processes or services.

25.4 WALKTHROUGH

25.4.1 Metrics: answering "How Much"

Logs tell you *what* happened; metrics tell you *at what scale* it is happening. In Effect, metrics are high-performance primitives that can be shared across your entire application.

```
import { Metric, Effect } from "effect"

// 1. Define a global counter
const OrderCount = Metric.counter("orders_processed").pipe(
  Metric.withConstantLabels({ service: "checkout" })
)

// 2. Increment it in your logic
const processOrder = (id: string) => Effect.gen(function* () {
  yield* Effect.log(`Processing order ${id}`)
  // ... work ...
  yield* Metric.increment(OrderCount)
})
```

25.4.2 Latency Histograms

Use histograms to track how long operations take. This is essential for detecting performance regressions in production.

```
const DbLatency = Metric.timer("db_query_duration")

const trackedQuery = dbQuery.pipe(
  Metric.trackDuration(DbLatency)
)
```

25.4.3 Zero-Overhead Semantic Observability

By defining metrics and spans as part of your program description, you ensure that observability is never "bolted on." It is a fundamental property of your system. If you change your logic, your spans and metrics naturally follow, following the principle of Law 7.

25.5 CHECKPOINT

- Observability is a semantic requirement of the program.
- Use Metrics for "what and how much."
- Use Logs for "who and why."
- Use Traces for "where and in what order."

25.6 EXERCISES

1. Add a root trace span to a complex multi-service workflow and use an observability tool (like Jaeger or Honeycomb) to visualize the call tree.
2. Convert an existing "latency measurement" that currently only logs a string into a `Metric.Histogram`.

3. Attach a business-relevant **Annotation** (like `orderId`) to a root span and verify that it appears in all child spans and logs.

25.7 FOOTGUNS

- **Prose over Data:** Writing long, conversational log messages instead of using structured annotations makes automated analysis impossible.
- **Metric State Leaks:** Using a global mutable variable to track counts instead of the provided `Metric` primitives, leading to incorrect values under concurrency.
- **Trace Disconnection:** Starting a new trace for every service call instead of propagating the parent span ID, making it impossible to see the end-to-end flow.

CANONICAL REFERENCE

- Logging
- Metrics
- Tracing

Chapter 26

EFFECT PLATFORM: THE "EDGE ALGEBRA"

26.1 PURPOSE

The "Edge Algebra" is the set of rules and tools for where your program finally meets the operating system. This chapter teaches you how to manage the absolute boundaries of your application, from subprocesses to production-grade entry points.

26.2 INVARIANTS

- **Inversion of Platform:** Business logic must never know about the concrete platform (Node vs. Browser); it must only know about the abstract service interfaces.
- **Managed Exit:** Production applications should always use `runMain` to handle signals and ensure all **Scope** finalizers run.
- **Typed Commands:** Subprocesses should be managed through the `Command` API to ensure type-safe interaction and robust error handling.

26.3 NEW CONCEPTS

- **Edge Algebra:** The collection of services and rules that govern the transition from program logic to OS execution.
- **Command:** A declarative description of a subprocess to be executed.
- **Terminal:** An abstraction for interacting with the standard input, output, and error streams.

26.4 WALKTHROUGH

26.4.1 Subprocess Pipelines

In standard Node.js, piping subprocesses is a manual nightmare of event listeners and stream buffering. In Effect, you can pipe `Command` objects as if they were simple functions.

```
import { Command } from "@effect/platform"

const listFiles = Command.make("ls", "-la")
const findTs = Command.make("grep", ".ts")

// Pipe stdout of listFiles to stdin of findTs
const pipeline = listFiles.pipe(
  Command.pipeTo(findTs),
  Command.string,
  Effect.map((s) => s.split("\n"))
)
```

26.4.2 Controlled Execution

Unlike `exec`, the `Command` API gives you absolute control over working directories, environment variables, and signal propagation.

```
const runInFolder = pipeline.pipe(
  Command.workingDirectory("./packages/core"),
  Command.env({ "DEBUG": "true" })
)
```

26.4.3 Interruption Propagation

If the parent fiber running a command is interrupted (e.g., because a timeout was hit), `Effect` automatically sends a `SIGTERM` to the underlying OS process. This ensures that you never leave "Zombies" running on your server. This is the OS-level fulfillment of Law 6.

26.5 CHECKPOINT

- Separate platform implementations from domain logic.
- Always use `runMain` for server and CLI tools.
- Use `Command` for all subprocess interactions.
- Ensure all platform requirements are satisfied in the `AppLayer`.

26.6 EXERCISES

1. Write a `Shell` service with a method `run(cmd: Command)` that executes a command and logs its output to the terminal.
2. Implement a program that uses your `Shell` service to run `npm --version` and fails if the version cannot be determined.

26.7 FOOTGUNS

- **Platform Leakage:** Importing from `@effect/platform-node` in a package that should be `platform-neutral`.

- **Silent Signal Failures:** Not using `runMain`, leading to applications that "hang" on Ctrl+C because they haven't registered the proper signal handlers.
- **Shell Injection:** Attempting to build command strings manually instead of using the `Command.make` arguments array.

CANONICAL REFERENCE

- Platform Introduction
- Command

Chapter 27

PLATFORM RUNTIME

27.1 PURPOSE

Effect is platform-agnostic by core, but real applications need to talk to the filesystem, the network, and the terminal. This chapter teaches you how to use **@effect/platform** to interact with these low-level APIs in a type-safe, testable, and consistent way across Node.js and the Browser.

27.2 INVARIANTS

- **Platform Abstraction:** Code should depend on interfaces like `FileSystem` or `HttpClient`, not directly on `node:fs` or `fetch`.
- **Managed Lifecycle:** Platform resources (like file handles or network agents) are always managed by `Scope`.
- **Edge-Only Runtimes:** Platform-specific runners (like `NodeRuntime.runMain`) should only appear at the application's entry point.

27.3 NEW CONCEPTS

- **Platform:** A set of service interfaces and implementations specific to an execution environment (Node.js, Browser, Bun).
- **FileSystem:** A high-level Effect service for interacting with files and directories.
- **HttpClient:** A composable, type-safe service for making HTTP requests.

27.4 WALKTHROUGH

27.4.1 Standardizing on abstractions

Instead of importing from `node:fs`, your logic should require the `FileSystem` service. This makes your code portable and easy to mock in tests.

```
import { Effect, FileSystem } from "@effect/platform"

const readConfig = Effect.gen(function* () {
  const fs = yield* FileSystem.FileSystem
  const content = yield* fs.readFileString("config.json")
  return JSON.parse(content)
})
```

27.4.2 Platform-specific wiring

To run the logic, you provide the implementation for your target platform. In Node.js, you use `NodeContext`.

```
import { NodeContext } from "@effect/platform-node"

const program = readConfig.pipe(
  Effect.provide(NodeContext.layer)
)
```

27.5 CHECKPOINT

- Depend on `@effect/platform` services for IO.
- Use platform-specific context layers at the entry point.
- Standardize on `HttpClient` for all network communication.
- Always use `runMain` for terminal/server applications.

27.6 EXERCISES

1. Build a program that reads a file named `VERSION` and logs its content, depending only on the `FileSystem` service.
2. Create a test layer that provides a mock `FileSystem` implementation and use it to verify your program without touching the disk.
3. Use `HttpClient` to fetch a JSON object from a mock URL and validate the response with a `Schema`.

27.7 FOOTGUNS

- **Direct Node Imports:** Importing from `node:fs` or `node:http` directly in your business logic makes your code impossible to run in the browser or test effectively.
- **Unmanaged Resource Handles:** Using raw file descriptors or network sockets without wrapping them in **Scope**, leading to resource leaks.
- **Ignoring Platform Errors:** Treating all IO failures as a generic `Error` instead of handling specific platform failure tags (e.g., `SystemError`).

CANONICAL DOCUMENTATION

Execution Capture: platform.runtime.runmain-graceful-teardown

NodeRuntime.runMain graceful teardown **NodeRuntime.runMain** runs your main fiber as the process entrypoint and ensures **Scope** finalizers run on SIGINT.

Status: Stable

Prereqs: Scope, Platform runtime

Last verified: 2026-01-30

WHEN TO USE

Use this for real Node programs where Ctrl-C should reliably interrupt work and trigger cleanup.

CANONICAL EXAMPLE

The listing is included directly from the repository. Run it and press Ctrl-C to observe the release path.

```
import { Effect } from "effect"
import { NodeRuntime } from "@effect/platform-node"

/**
 * Run this file directly and press Ctrl-C.
 *
 * The point of the example is that:
 * - resources acquired in a Scope are deterministically released
 * - the release happens even when the process is interrupted by
 *   SIGINT
 */
const program = Effect.scoped(
  Effect.gen(function* () {
    const interval = yield* Effect.acquireRelease(
      Effect.sync(() => {
        console.log("acquire: ␣start␣interval")
        return setInterval(() => console.log("tick"), 1000)
      }),
      (handle) =>
        Effect.sync(() => {
          console.log("release: ␣clear␣interval")
          clearInterval(handle)
        })
    )

    // keep the program alive until interrupted
    yield* Effect.never

    // NOTE: unreachable
    return interval
  })
).pipe(
  Effect.ensuring(
    Effect.sync(() => {
      console.log("ensuring: ␣ran␣(after␣scope␣closes)")
    })
  )
)

NodeRuntime.runMain(program)
```

WHAT TO NOTICE

- The interval is acquired via **Effect.acquireRelease**

Chapter 28

TESTING: MAKE TIME, RESOURCES, AND FAILURES DETERMINISTIC

28.1 PURPOSE

Testing in Effect is not just about asserting outputs; it is about controlling the environment. This chapter teaches you how to leverage Effect's laziness and environment-awareness to make time, resources, and complex service dependencies completely deterministic in your test suite.

28.2 INVARIANTS

- **Environment Control:** Tests must swap real-world implementations (Live layers) for deterministic test implementations via the `R` channel.
- **Deterministic Time:** All time-based operations (`sleep`, `timeout`, `retry`) must be verified using the `TestClock` to avoid slow, flaky tests.
- **Scoped Verification:** Resources acquired during a test must be managed by `it.scoped` to guarantee cleanup even upon assertion failure.

28.3 NEW CONCEPTS

- **it.effect:** A Vitest wrapper that automatically runs an Effect and handles errors and context.
- **TestClock:** A simulated clock that allows you to "fast-forward" time in tests.
- **it.scoped:** A test wrapper that provides a local `Scope` for resource management.

28.4 WALKTHROUGH

28.4.1 Standardizing on @effect/vitest

The `@effect/vitest` library is the standard because it manages the runtime for you. It automatically configures the `TestClock` and provides a fresh `Scope` for every test.

28.4.2 Deep Mocking: Swapping layers

Suppose you have a complex application layer. In a specific test, you want to verify how the system handles a database error. You don't need to change your application logic; you just swap the database implementation at the test boundary.

```
import { it } from "@effect/vitest"
import { Effect, Layer } from "effect"

const DbMock = Layer.succeed(Database, {
  save: () => Effect.fail(new DbError("Simulated failure"))
})

it.effect("handles DB failure gracefully", () => Effect.gen(function*
() {
  const result = yield* runComplexLogic.pipe(
    Effect.provide(DbMock), // Swap implementation
    Effect.provide(AppLayer) // satisfy remaining dependencies
  )
  expect(result.status).toBe("error")
}))
```

28.4.3 Controlling Time with TestClock

Never use `vi.useFakeTimers()`. Instead, use `TestClock.adjust`. It allows you to skip time instantly and deterministically.

```
it.effect("retries every minute", () => Effect.gen(function* () {
  const fiber = yield* taskWithRetry.pipe(Effect.fork)

  // 1. Fast forward 1 minute
  yield* TestClock.adjust("1 minute")

  // 2. Check the logs
  const logs = yield* getLatestLogs()
  expect(logs).toContain("Retrying...")

  yield* Fiber.interrupt(fiber)
}))
```

28.5 CHECKPOINT

- Standardize on `@effect/vitest` for all tests.
- Use `it.effect` to automatically run program blueprints.
- Never use real timeouts in tests; use `TestClock`.
- Provide mock implementations via the **Layer** graph.

28.6 EXERCISES

1. Write a test for a retry policy that uses exponential backoff. Use `TestClock` to verify it retries at the correct intervals without actually waiting.
2. Create a `Layer.scoped` for a mock service and verify that its cleanup logic runs when the test completes.
3. Use `it.effect.fails` to document a known edge case that currently results in a failure.

28.7 FOOTGUNS

- **Real Time Leaks:** Forgetting to use `it.effect` and using a standard `it` block, which causes `Effect.sleep` to actually pause the test runner.
- **Mock Bloat:** Creating overly complex mock implementations that require their own tests; keep mocks simple and deterministic.
- **Swallowing Errors:** Using `Effect.ignore` or similar operators in tests, which can hide assertion failures.

VERIFICATION RESULTS

When run, the test suite produces:

```
> effect-bible@ test /Users/af/effect-bible-expanded-v2
> vitest run --reporter=verbose --no-color

RUN   v4.0.18 /Users/af/effect-bible-expanded-v2

PASS packages/bible-examples/test/scope.test.ts > Scope /
  acquireRelease > runs the finalizer when the local scope closes 18
  ms
PASS packages/bible-examples/test/pricing.test.ts > Pricing.getPrice >
  applies surge pricing when enabled 20ms
PASS packages/bible-examples/test/pricing.test.ts > Pricing.getPrice >
  returns base price when disabled 6ms
PASS packages/bible-examples/test/vitest-demo.test.ts > @effect/vitest
  core patterns > is just a value (basic pattern) 21ms
PASS packages/bible-examples/test/vitest-demo.test.ts > @effect/vitest
  core patterns > tests a long delay instantly (TestClock) 19ms
PASS packages/bible-examples/test/vitest-demo.test.ts > @effect/vitest
  core patterns > uses the real clock (Live pattern) 12ms
PASS packages/bible-examples/test/vitest-demo.test.ts > @effect/vitest
  core patterns > provides a test-level scope (Scoped pattern) 6ms
PASS packages/bible-examples/test/deferred.test.ts > Deferred > lets
  one fiber signal readiness to another 56ms

Test Files  4 passed (4)
```

```
Tests      8 passed (8)
Start at   17:57:46
Duration   461ms (transform 169ms, setup 0ms, import 1.06s, tests 161
          ms, environment 0ms)
```

CANONICAL REFERENCE

- Vitest Integration
- TestClock
- TestRandom

Tier VI

THE HORIZON

CONTENTS

29 The Way of the Architect: Toward a Post-Promise Era	131
29.1 Purpose	131
29.2 Invariants	131
29.3 The Horizon Concepts	131
29.4 Walkthrough: The Architecture of the Next Decade	132
29.4.1 The AI-Human Hybrid Lifecycle	132
29.4.2 Post-Framework Composition	132
29.5 The Architect’s Manifesto	133
30 Glossary of Intent	135
Addendum: Essential Resources	137
Index	143

Chapter 29

THE WAY OF THE ARCHITECT: TOWARD A POST-PROMISE ERA

29.1 PURPOSE

We stand at the precipice of a fundamental shift in how software is conceived and constructed. For decades, we have been trapped in an era of "Imperative Hope"—writing instructions and hoping that our tests catch enough edge cases to prevent a production disaster.

The transition from a Developer to an **Effect Architect** is the final departure from this paradigm. This chapter is your roadmap for the next decade of development. It explores a world where frameworks become implementation details, where AI becomes a high-precision tool for logic synthesis, and where the "Distributed Fiber" makes the boundaries of a single machine irrelevant. You are no longer just writing code; you are designing *Verified Blueprints of Intent*.

29.2 INVARIANTS

Architectural integrity in the next decade will be defined by three unwavering properties:

- **Mathematical Honesty:** The type signature must remain a total and complete description of the program's potential outcomes.
- **Total Supervision:** Every computational fiber must exist within a hierarchy that guarantees its finalization.
- **Systemic Control:** The architecture must be self-healing and self-supervising.

29.3 THE HORIZON CONCEPTS

- **Total Reification:** The process of turning implicit system concerns—like retries, auth, logging, and timeouts—into first-class data values (`Schedules`, `Context`, `Spans`) that can be manipulated and composed.

- **AI-Native Blueprint:** A paradigm where human architects define the "Obligation Contract" (R and E) and AI models synthesize the "Success Logic" (A) within those safe bounds.
- **Distributed Fiber:** A future runtime where a Fiber's execution context is mobile across different physical nodes, enabling true "Cloud-Native" code without boilerplate networking logic.
- **Post-Framework Architecture:** A state where libraries like Express, React, or Nest.js are relegated to being "Input/Output Drivers" provided at the Composition Root, rather than the defining structure of the application.

29.4 WALKTHROUGH: THE ARCHITECTURE OF THE NEXT DECADE

29.4.1 The AI-Human Hybrid Lifecycle

In the post-Promise era, the human's role shifts from "writing logic" to "defining obligations."

1. **The Human Architect:** Defines the `Context.Tag` and the `Error Algebra`.
2. **The AI Synthesis:** Generates the `Effect.gen` implementation that satisfies those tags and handles those errors.
3. **The Compiler:** Verifies that the AI's logic doesn't leak unhandled tags or catch-all errors.

```
// HUMAN DEFINITION (The "Constraint")
export interface PaymentService {
  readonly charge: (amount: number) => Effect<Receipt,
    InsufficientFunds | NetworkError>
}

// AI SYNTHESIS (The "Logic")
// The AI might try to write complex logic, but it MUST
// return exactly what the Human specified.
```

29.4.2 Post-Framework Composition

Today, your choice of framework (e.g., NestJS) often dictates your code's structure. In the future, the framework is just another **Layer**.

```
// Logic is pure and platform-blind
const businessProcess = Effect.gen(function*() { ... })

// Framework is provided as an adapter at the very end
const runnable = businessProcess.pipe(
  Effect.provide(ExpressHttpAdapterLive),
  Effect.provide(PostgresLive)
)
```

This allows you to move your business logic from a Lambda to a long-running Kubernetes pod—or even to the Edge—by changing exactly **one line** in your Composition Root.

29.5 THE ARCHITECT'S MANIFESTO

As you close this manual and return to your terminal, remember that you are no longer a victim of the "Async/Await" void. You have been given the tools to build systems that endure.

1. **Be Honest:** Ensure every failure is represented in the E channel. Never lie to your callers with a generic Error or any.
2. **Be Pure:** Keep your dependencies in R. Let the world be provided, not hardcoded.
3. **Be Lazy:** Never perform work until you are at the edge. Preserve the "Blueprint" at all costs.
4. **Be Supervised:** Ensure every concurrent task has a parent. Leave no "Orphan Fiber" behind.

The Ten Laws are not constraints on your creativity; they are the **foundations of your freedom**. By surrendering to the discipline of "Architecture by Construction," you gain the freedom to build systems of infinite complexity with zero fear.

BUILD WITH POWER.

Targeting Effect 3.19.14

Chapter 30

GLOSSARY OF INTENTS

This glossary provides authoritative, high-density definitions for the core vocabulary of the Effect ecosystem.

Cache A service for storing the results of Effectful computations to avoid redundant re-execution. Effect provides a built-in Cache module with support for expiration and size limits.

Cause A data structure representing all possible ways an Effect can fail, including expected errors, defects (unexpected errors), and interruptions. It preserves the full graph of concurrent failures.

Chunk An immutable, high-performance array-like data structure used extensively within Effect for collections of values. It supports efficient concatenation and slicing.

Config A description of a configuration value (like an environment variable) that includes its type, validation logic, and default value.

Context An immutable map derived from Tags to their service implementations. It represents the “environment” or requirements (R) of an Effect.

Data A module providing utilities for value equality and immutable data structures. `Data.Struct` and `Data.TaggedError` are canonical tools for domain modeling.

Defect An unexpected error that is not tracked in the error channel (E) of an Effect. Defects represent bugs or catastrophic failures (like `throw`).

Deferred A synchronization primitive that represents a value that may not yet be available. It can be completed exactly once, and fibers can wait for its completion.

Effect A lazy, immutable description of a computation that can fail, require requirements, and return a value. The blueprint of intent.

Exit The final result of an Effect’s execution—either **Success** with a value or **Failure** with a Cause.

Fiber A lightweight “logical thread” managed by the Effect runtime. Thousands of fibers can run on a single OS thread.

Layer A recipe for constructing a Service. Layers enable modular, type-safe dependency injection.



SCAN FOR DOCS

Scope A lifetime boundary for resources. Resources acquired within a scope are automatically released in LIFO order when the scope closes.

Runtime The state machine that executes Effects by stepping through their instructions and managing fiber scheduling.

Schema A dual-purpose definition that enables both runtime validation and TypeScript type inference. The boundary of trust.

Stream A lazy description of a computation that produces a sequence of zero or more values, supporting backpressure and resource management.

Tag A unique identifier for a service, used to retrieve it from the **Context**. It is the key in the dependency injection map.

ADDENDUM: ESSENTIAL RESOURCES

The journey to architectural mastery does not end here. The Effect ecosystem is deep, evolving, and supported by a vibrant community of engineers. Use these resources to stay at the cutting edge.

THE HUBS

effect.website

The primary portal for the ecosystem. Home to the official blog, community announcements, and the gateway to the Discord server.

effect.website/docs

The official documentation. From beginner tutorials to advanced guides on Stream and Schema, this is the definitive technical reference. Includes the **API Reference** for every package in the monorepo.

PRODUCTION & PATTERNS

effect.solutions

A prescriptive, high-level guide focused on production-ready patterns. It provides idiomatic "Real World" solutions for common architectural challenges like data modeling, configuration, and testing.

GitHub: Effect-TS

The source of truth. Explore the main monorepo (`effect`) to see the implementation of the core library, or specialized repositories for AI, CLI, RPC, and more.

EDUCATION & MASTERY

effect.institute

The educational arm of the ecosystem. Offers structured learning paths, deep-dive courses, and certification programs designed to take developers from "curious" to "architect" through rigorous, hands-on training.

”The Architect does not build the building; they build the system that builds the building.”
— Meta-Invariant of the α Edition

SOURCE DRIFT VERIFICATION

```
[verify-sources] OK effect.website/llms-full.txt
[verify-sources] optional source not vendored: effect.solutions/home
[verify-sources] optional source not vendored: effect.solutions/
services-and-layers
[verify-sources] optional source not vendored: effect.solutions/testing
[verify-sources] optional source not vendored: effect.solutions/error-
handling
[verify-sources] optional source not vendored: effect.solutions/config
[verify-sources] optional source not vendored: effect.institute/og/home
[verify-sources] optional source not vendored: effect.institute/
changelog
```

TRACKED SOURCES

See `book/vendor/sources.json` for the full list of canonical endpoints tracked by this build pipeline.

APPENDIX: SOURCE DRIFT

See `book/vendor/` and `book/scripts/verify-sources.mjs`.

INDEX

A, 9, 16, 42, 95, 132
acquireRelease, 49
Adapter Package, 73
AI-Native Blueprint, 132
Annotation, 107, 113
App Package, 73
AppLayer, 57

Backpressure, 95

Cache, 135
Catching, 33
Cause, 33, 34, 135
Cause, 135
Chunk, 135
Command, 115
Command, 115, 116
Composition Root, 57
Config, 61, 135
Config, 61, 63
ConfigError, 62
ConfigProvider, 61
Constructor, 45
Context, 41, 135, 136
Context, 53, 131
Context.Tag, 58, 132

Data, 135
Database, 42
Date, 65, 67
Defect, 33, 135
Deferred, 90, 91, 135
Deferred, 91–93
Deferred.await, 90

Deferred.fail, 90
Deferred.succeed, 90
Deferred<A,E>, 90
Distributed Fiber, 132
Distributed Tracing, 111

E, 16, 28, 34, 35, 132, 133
Edge Algebra, 115
Effect, 7, 135
Effect, 55, 57
Effect.acquireRelease, 122
Effect.gen, 132
Effect.race, 90
Effect.scoped, 122
Effect<A, E, never>, 20
Effect<A, E, R>, 5
Effect<A, never, R>, 20
Error Algebra, 132
Execution Boundary, 53
Exit, 135
Exit, 17

Feature, 69
Fiber, 5, 81, 82, 135
FiberRef, 122
FileSystem, 119
FileSystem, 119, 120
Finalizer, 49
Forking, 82

HttpClient, 119
HttpClient, 51, 119, 120

Internal Package, 73

- Interruption, 82
- it.effect, 123
- it.scoped, 123
- Joining, 82
- Latch, 91
- Layer, 41, 45, 50, 61, 69–71, 99, 100, 124, 132, 135
- Layer, 47, 57
- Logger, 107
- ManagedRuntime, 53
- ManagedRuntime, 54, 55
- Memoization, 45
- Metric, 111
- Metric, 113
- Metric.Histogram, 112
- NodeRuntime.runMain, 122
- Orchestration, 69
- Parse, 65
- ParseError, 67
- Platform, 119
- Post-Framework Architecture, 132
- Preservation of Intent, 17
- Promise, 7
- PubSub, 101
- PubSub, 92
- Queue, 99
- Queue, 92, 99, 100
- Queue.sliding, 101
- R, 9, 16, 41–43, 71, 123, 132, 133
- R=never, 41
- Redacted, 61
- Ref, 99
- Ref, 99–101
- Retry, 85
- Runtime, 53, 54, 136
- Runtime, 53
- Schedule, 85
- Schedule, 86
- Schedule.spaced, 122
- Schedules, 131
- Schema, 8, 65, 66, 69, 71, 136
- Schema, 65, 74, 120, 137
- Scope, 8, 49, 50, 83, 96, 115, 119, 120, 122, 123, 136
- Scope, 50, 51, 123
- Semantic Logging, 111
- Semaphore, 91, 92
- Semaphore, 92, 93
- Service, 41
- Service, 135
- Sink, 95, 97
- Slice, 69
- Span, 107
- Spans, 131
- Stream, 95, 136
- Stream, 95, 96, 137
- Stream.acquireRelease, 97
- SynchronizedRef, 99
- SynchronizedRef, 100
- Tag, 41, 136
- Tagged Error, 33
- Terminal, 115
- TestClock, 123
- Timeout, 85
- Total Reification, 131
- Transformation, 65
- UserRepo, 70
- while loop, 83
- Wiring, 57